



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.2: Network Slicing

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

Advanced Network Application Paradigms

This 27-hour module is a deep dive into the next generation of network technologies, structured into four key topics that explore how networks are being transformed to support modern, demanding applications.

Part 1: Mobile Edge Computing (MEC) (7 hours)

- This topic explores **Mobile Edge Computing (MEC)**, a distributed paradigm that brings cloud capabilities to the network's edge. We'll see how MEC drastically reduces latency by processing data locally, enabling real-time applications like **AR**, **VR**, and **video analytics** that are impossible with traditional centralized clouds.

Part 2: Network Slicing (7 hours)

- This topic explores **Network Slicing**, a key 5G technology that partitions a single physical network into multiple virtual "slices." Each slice can be customized for specific performance requirements like bandwidth and latency, serving applications from **IoT devices** to **autonomous vehicles**.

Part 3: Multimedia Networking (7 hours)

- This topic is dedicated to the challenges of delivering high-quality multimedia content. We'll cover protocols and techniques for ensuring quality of service (QoS) and managing jitter and latency. Key topics include **adaptive bitrate streaming**, **WebRTC**, and the role of **CDNs**.

Part 4: Distributed Machine Learning (DML) (6 hours)

- This topic explores **Distributed Machine Learning (DML)**, where ML models are trained and run across a network of devices. It enables local computation, reducing reliance on a central server. We'll discuss **federated learning**, where a model is trained on devices without sharing raw data, and its benefits for **personalized assistants** and **IoT analytics**.

Motivations



Foundations of Network Slicing

In Part 1, we learned how to deploy applications at the edge of the mobile network.

Now, we ask a different question: **How can the network itself be fundamentally reshaped to meet the diverse and often conflicting demands of these new applications?** capacità di banda vs latenza => voglio una sola rete con la capacità di risolvere diverse richieste

This part introduces **Network Slicing**, the 5G paradigm for **transforming a monolithic physical network into a versatile platform** capable of hosting multiple, isolated, fit-for-purpose logical networks.

piattaforma + versatile pensata per diverse applicazioni
=> non più approccio monolitico => dove la rete cerca
di soddisfare tutte le richieste anche opposte fra di loro!

The Problem: The "One-Size-Fits-All" Network

Previous generations of mobile networks (2G/3G/4G) were built with a monolithic, "one-size-fits-all" approach. They were primarily designed and optimized for a single service: mobile broadband for human users.

This created a fundamental conflict when trying to support new, diverse services:

- An autonomous car requires **ultra-low latency**, but a video stream needs **high bandwidth**.
- A massive IoT deployment needs to support **millions of low-power devices**, while a gamer needs a **stable, low-jitter** connection.

Trying to serve all these needs on a single, statically configured network leads to compromises where no application gets the performance it truly requires.

si va a trovare il compromesso che in realtà alla fine non riesce a soddisfare nessuno

The Vision of Network Slicing

The core idea of Network Slicing is to move away from a single, rigid network.

The Vision: To partition a single physical network infrastructure into multiple, independent, end-to-end virtual networks. Each of these virtual networks, or "slices," is custom-built to meet the specific requirements of a particular application or customer.

The Analogy: Network Slicing is to **networking** what **Virtual Machines** are to **compute**. A single powerful physical server can be partitioned to run multiple, isolated VMs, each with its own dedicated resources (CPU, RAM) and operating system. Similarly, a single physical network can be partitioned to run multiple, isolated network slices.

come costruite macchine virtuali ad-hoc => una sua cpu, ram , sis operativo ecc... => lo stesso per rete

allochiamo tante
reti virtuali
costruite in base
ai req delle app.

What is a Network Slice? The 3GPP Definition

A network slice is not just a VPN or a QoS queue. It is a complete, self-contained, and programmable logical network.

3GPP Definition: "A Network Slice is a logical network that provides specific network capabilities and network characteristics." si va verso la possibilità di fornire determinati requisiti a seconda dell'applicazione => piuttosto che solo il QoS

Deconstructing the Definition:

- **"Logical network"**: It's realized in software and configuration, **not new physical hardware.** si cerca di usare hardware general purpose per realizzare una rete logica ma software => non installo una nuova rete fisica per un determinato servizio
- **"Specific network capabilities"**: It can be composed of a custom set of network functions (e.g., **a slice for IoT might not need a voice service function**). creo rete per quel part. servizio
- **"Specific network characteristics"**: It is configured to deliver a guaranteed level of performance (e.g., 10ms latency, 99.999% reliability). e per quelle performance richieste
- **Crucially, it is End-to-End**: A slice **spans all domains of the network,** from the user's device (UE) to the application server. si considerano tutti i segmenti della rete dall'utente all'altro => segmentazione logica(?)

The Four Pillars of Network Slicing

The paradigm is built on four essential principles:

se considero net slicing => non posso limitarmi a garantire determinate prestazioni in un pezzetto della rete => ma questo deve essere garantita end-to-end

- 1. End-to-End (E2E) Nature:** A slice is a complete logical network, including the Radio Access Network (RAN), the Transport Network (TN), and the Core Network (CN). You cannot achieve an E2E latency guarantee by only managing the Core.
- 2. Isolation:** Slices are isolated from each other in terms of resources, performance, and security. A failure or traffic storm in one slice must not affect another.
- 3. Customization:** Each slice can have a custom topology, a tailored set of network functions, and unique policies for security, mobility, and QoS.
- 4. Automation:** Slices are designed to be managed and orchestrated automatically. They can be created on-demand, scaled, and terminated via software APIs, often in minutes. slices gestite in maniera autonoma una rispetto all'altra

i parametri della rete possono essere modificate

The Business Driver: From Connectivity to NaaS

Network Slicing is a fundamental shift in the business model of telecom operators.

Old Model: Sell a single product—mobile connectivity—billed by the Gigabyte.

si guardava quanto
traffico potesse
gestire

New Model (NaaS): Sell customized, end-to-end **Network-as-a-Service** products to different "vertical" industries.

Examples of NaaS Products:

ultra reliable low latency

- "The Automotive Slice": A premium, high-cost slice offering URLLC for connected car safety functions.
- "The Logistics Slice": A low-cost, wide-area slice optimized for mMTC to track shipping containers.
- "The Media Slice": A high-bandwidth eMBB slice for a broadcasting company to stream a live sporting event.

massive- machine type communication

Network Slicing: Historical Background

idea di creare un'unica infrastruttura di rete ma che risponde a diversi servizi è nata da un po'...

Overlay Networks (1980s):

- First example of network slicing.
- Combined heterogeneous network resources to create virtual networks.

Evolution to VLANs and VXLANs:

- Provided virtual networks over common infrastructure.
- Lacked programmability.

PlanetLab (1990s)

- Introduced virtualization framework for programmable slices
mancavano le funzionalità delle reti virtuali e della softwarizzazione

Modern Network Slicing:

- Enabled by network virtualization and resource abstraction.
- Real-world programmable network slices are now feasible.

Network Slicing Enablers

Network slicing in 5G is a key feature that allows the creation of multiple virtual networks on a shared physical infrastructure.

Software Defined Networking (SDN):

- Centralized control of network resources.
- Dynamic and flexible network management.

ho la capacità di gestire in maniera dinamica il traffico e le risorse di rete

Network Function Virtualization (NFV):

- Decoupling of network functions from hardware.
- Deployment of network services as virtualized instances.

posso decidere in maniera dinamica quali sono i servizi di cui ho bisogno per quella diversa rete

Cloud Computing:

- Scalable and on-demand computing resources.
- Support for diverse and resource-intensive applications.

questa mi serve per gestire tutti le diverse infrastrutture di computing

Edge Computing:

- Processing data closer to the source.
- Reduced latency and improved real-time performance.

va a sostituire il cloud quando può e quando riesce

Slicing vs. QoS: A Technical Distinction

It's a common misconception to think of slicing as just "QoS on steroids." They are fundamentally different concepts.

Traditional QoS (e.g., DiffServ):

- **Mechanism:** **Prioritizes packets** within a **single, shared pipe**. It marks some packets as more important (e.g., VoIP traffic) so that during congestion, they are less likely to be dropped. *agisce solamente in un particolare dominio IP, risente comunque di problemi di traffico*
- **Limitation:** **It only provides relative priority**. It cannot provide a *guaranteed* outcome, and a massive flood of low-priority traffic can still impact high-priority traffic. It doesn't isolate network functions.

Network Slicing:

- **Mechanism:** Creates multiple, **parallel, logically separate pipes**. Each slice is allocated its own dedicated (or guaranteed minimum) share of radio, transport, and compute resources.
- **Benefit:** **Provides true performance isolation**. A traffic storm in the "Best-Effort Internet" slice has no impact on the guaranteed latency of the "Critical Robotics" slice. Each slice can also have its own custom network functions.

*slice come qualcosa di + => in quanto fornisce anche particolari risorse
=> le quali sono gestite in maniera dinamica e flessibile!*

Slicing vs. VPN: A Technical Distinction

VPN (Virtual Private Network):

- **Purpose:** Primarily provides **security and privacy** by creating an **encrypted tunnel over a public network** (like the internet).
- **Layer:** Typically operates at Layer 3 (IPsec) or Layer 2 (L2TP).
- **Performance:** A VPN has **no inherent performance guarantees**. Its performance depends entirely on the underlying, best-effort network.

Network Slice:

- **Purpose:** To provide a complete logical network with **guaranteed performance characteristics (latency, throughput, reliability)** and **custom functionality**.
- **Layer:** Is an end-to-end concept spanning all layers, from the physical radio (Layer 1) up to the application functions (Layer 7).
- **Security:** A slice provides security through isolation, **but a VPN can still be run inside a slice for an extra layer of encryption.**
tipo vpn all'interno comunque della mia slice => tubo di riservatezza fra le reti
qui non si vanno a guardare i requisiti prestazionali => unico obiettivo è quello di garantire riservatezza della rete

Slicing vs. Network Virtualization (NFV)

NFV (Network Functions Virtualization) is an **enabling technology**. It provides the ability to run network functions (like a UPF or SMF) as software on COTS hardware. NFV provides the "what" (the virtualized functions).

Network Slicing is an **end-to-end service architecture**. It *uses* NFV to create, *manage*, and chain together the specific network function instances needed to build a complete, isolated logical network. Slicing is the "how" (the E2E service composition).

You cannot have dynamic, on-demand network slicing without the flexibility of NFV and cloud-native principles.

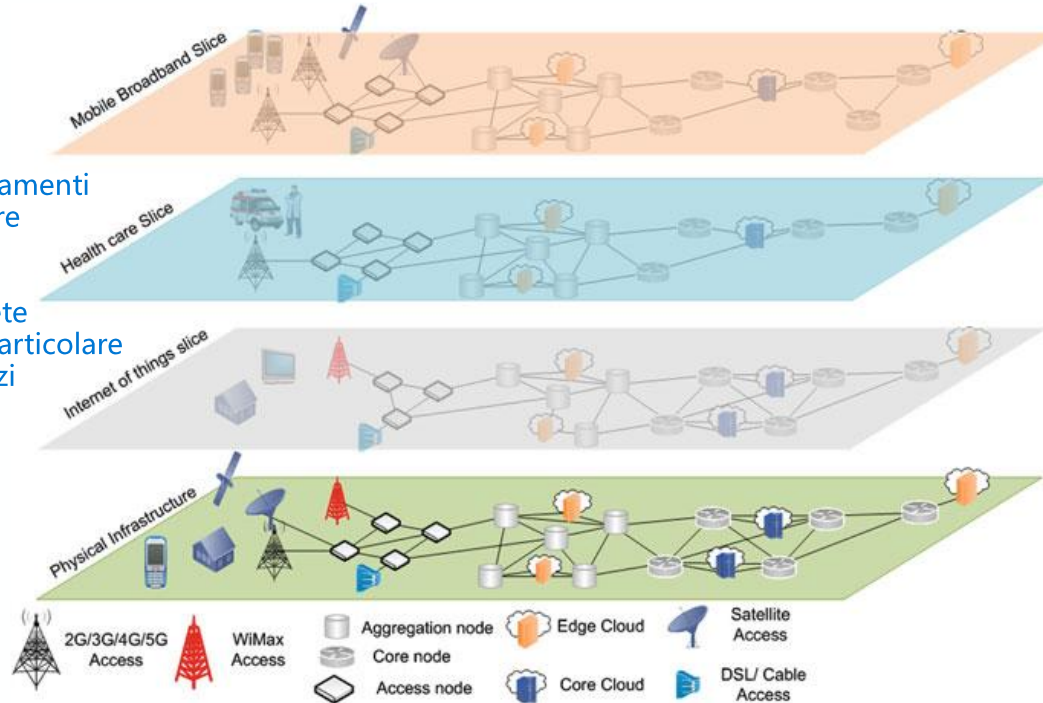
è comunque qualcosa di + rispetto al solo NFV => basti pensare ai requisiti end-to-end della rete

Architecture and Terminology



System view of Network Slicing

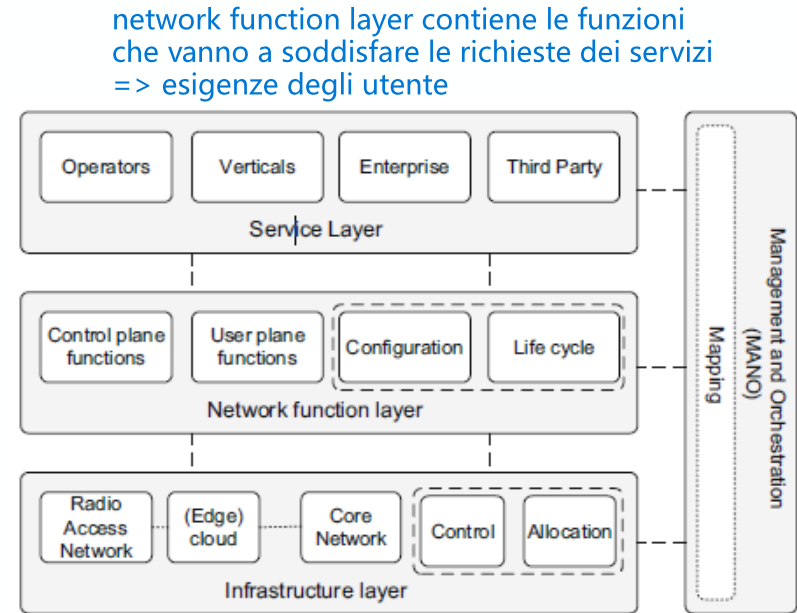
riservo altri collegamenti
per quel particolare
servizio
=> segmento
infrastruttura di rete
ognuna per una particolare
esigenza dei servizi



Network Slicing Layered Architecture

This framework presents a three-layer architecture for network slicing, along with a management and orchestration (MANO) entity:

- **Service Layer:** services and applications provided by the network.
- **Network Function Layer:** configuration and lifecycle management of network functions.



Network Slicing Layered Architecture

This framework presents a three-layer architecture for network slicing, along with a management and orchestration (MANO) entity:

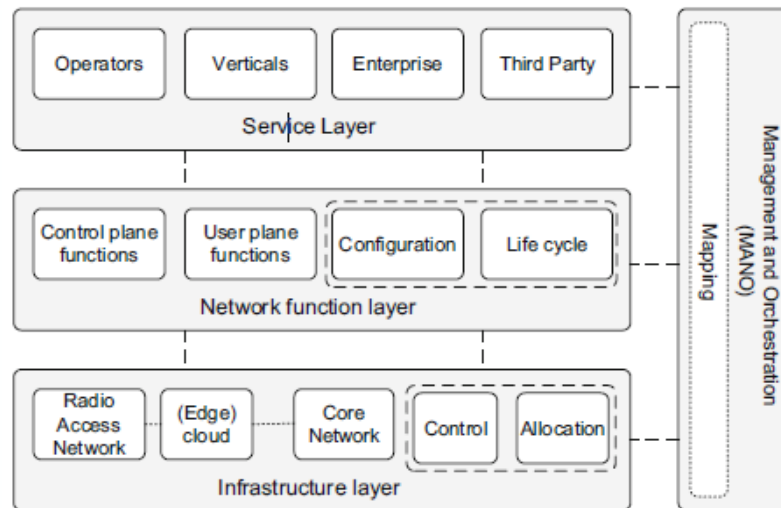
qui si vanno a mappare le funzioni usate/ richieste

- **Infrastructure Layer:** physical network infrastructure, including both the core network and the radio access network.

quello che va a gestire il tutto => lui sa come mappare una determinata

- **Management and Orchestration (MANO) Entity:** responsible for translating service models and use cases into network slices.

funzione ad un particolare servizio (anche in maniera condivisa) e come allocare le infrastrutture per una determinata funzione



Network Slicing technologies

Network Functions (NFs):

- Express elementary network functionalities.
- Used as building blocks to create each network slice.

Virtualization: quello che permette la flessibilità per la creazione dei servizi
=> altrimenti dovrei fare tutto in maniera rigida!

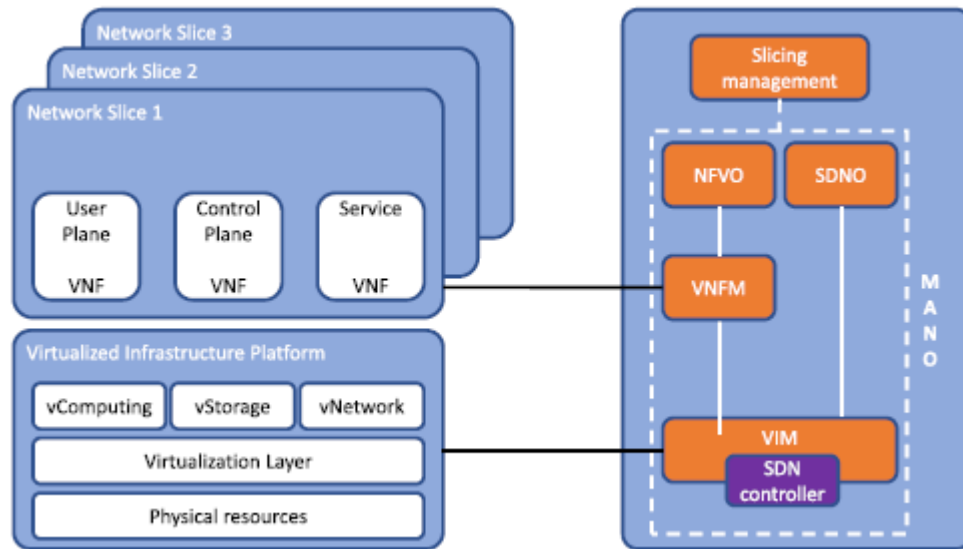
- Provides an abstract representation of physical resources under a unified scheme.
- Enables scalable slice deployment using Network Function Virtualization (NFV).
- Allows decoupling of each network function instance from the network hardware it runs on.

Orchestration:

- Coordinates all network components involved in the life cycle of each network slice.
- Uses Software-Defined Networking (SDN) to enable dynamic and flexible slice configuration.

The generic architecture for network slicing

punto di vista + implementativo/architetturale => si vedono i diversi costituenti



orchestratore
per le diverse funzioni
uno per le funzioni virtuali
e uno per la gestione
dell'infrastruttura
infine è aggiunta quella
per gestire le diverse slice
=> ho la possibilità di
gestire
contemporaneamente i
diversi servizi che magari
possono operare insieme
=> diversi sottoelementi
della rete stessa

GSMA => associazione degli operatori
rendere interoperabili i profili d'uso forniti dal
3GPP => ha definito una serie di templates per
standardizzare i casi d'uso prima => eMBB,
URLLC, mMTC

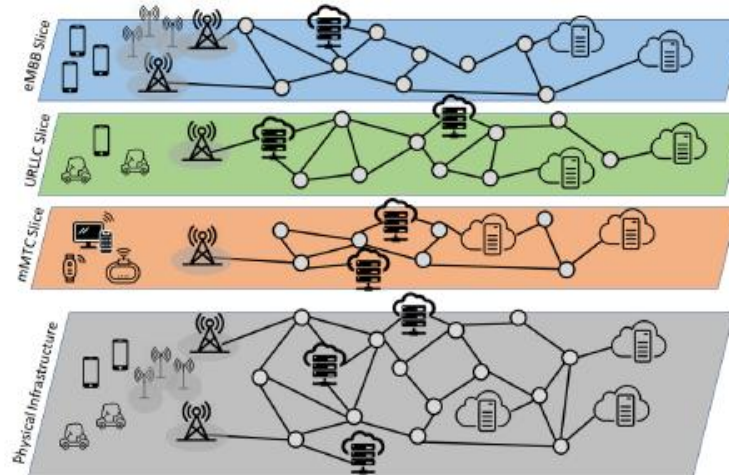
GSMA Generic Slice Templates (GST)

To simplify the ordering and creation of slices, the **GSMA** has defined a set of **Generic Slice Templates (GSTs)**.

A GST is a standardized blueprint that defines the characteristics and performance attributes for a common type of service.

The three primary GSTs correspond directly to the main 5G use cases. We will examine each.

1. Slice Type 1: eMBB
2. Slice Type 2: URLLC
3. Slice Type 3: mMTC



del tipo se vuoi creare
una slice di un certo tipo
devi andare a rispettare
determinati requisiti
(minimi)

Slice Type 1: eMBB

inizialmente goal principale

This slice is optimized for applications that require **high data rates**.

Key Characteristics:

- **High throughput (Gbps).**
- Medium to low latency. *non troppo alta*
- Wide area coverage. *per gestire utenti in movimento*

Technical Implementation: *bande + larghe*

- **RAN:** **Uses wide channel bandwidths**, advanced MIMO, and scheduling algorithms that favor throughput. *accesso radio in cui si utilizzano tante antenne contemporaneamente*
- **Core:** **UPFs with high forwarding capacity.** *user-plane => per garantire e2e
=> sia in trasmissione che in ricezione => tanti canali in parallelo => dunque alto throughput*

Use Cases: 4K/8K video streaming, **Fixed Wireless Access (FWA)**, immersive AR/VR experiences.

fwa come slice => banda garantita

=> non deve competere con altri utenti => come nel caso del 5G

Slice Type 2: URLLC

This slice is designed for mission-critical applications where **latency and reliability are paramount**.

Key Characteristics:

- Ultra-low latency (<5ms).
- Extremely high reliability (99.999% or better).
- Moderate throughput.

Technical Implementation:

aggiungo stazioni radio => evito congestioni

- **RAN:** Uses mini-slot scheduling, redundant radio bearers, and other specialized radio features. partiziono i dati in modo da inviarli in maniera veloce => ma anche leggerli
- **Core:** Often requires deploying UPFs and application servers at the extreme edge (MEC). riduco i tempi di trasmissione/ricezione

Use Cases: Industrial robotics, autonomous vehicles (V2X), remote surgery, smart grid control.

Slice Type 3: mMTC

This slice is optimized to support a **massive number of simple, low-power devices**.

Key Characteristics:

- High connection density (>1 million devices/km²).
- Extremely low power consumption (enabling 10+ year battery life).
- Low data rates and high latency tolerance.

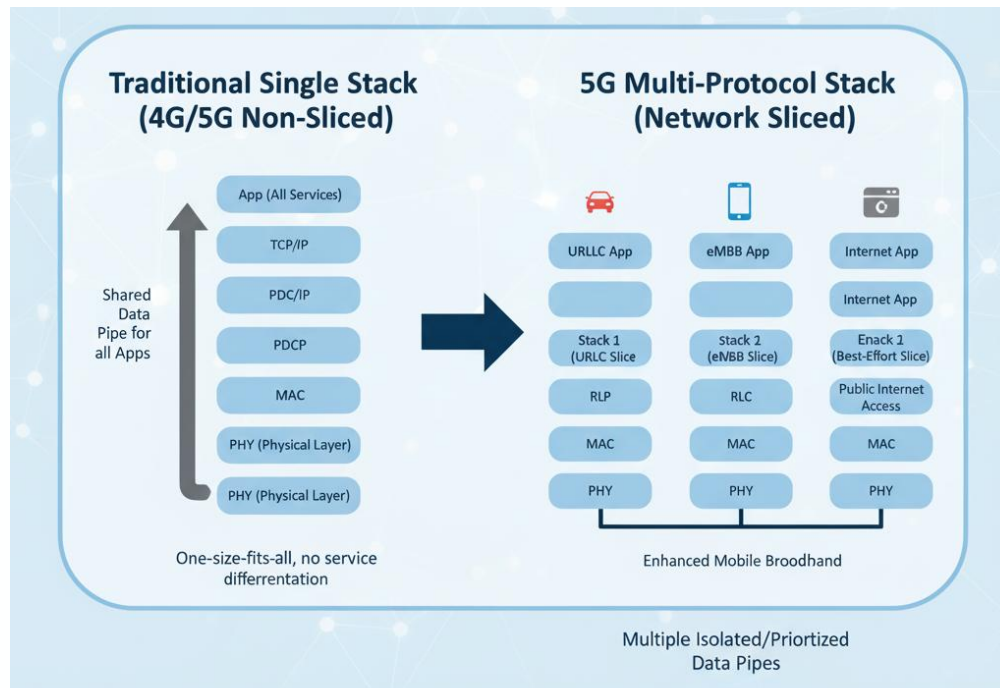
Technical Implementation:

trasmettitore spesso in idle

- **RAN:** Uses narrowband technologies (like NB-IoT) and power-saving modes (PSM, eDRX).
- **Core:** Network functions are optimized to handle a huge number of simultaneous, small sessions efficiently.

Use Cases: Smart meters, environmental sensors, asset tracking, smart agriculture.

Protocol Stack View of Slicing



The Concept of Isolation

Isolation is the most critical and defining feature of network slicing. It is the guarantee that the resources, performance, and security of one slice are protected from the activities of all other slices sharing the same physical infrastructure.

Without strong isolation, a network slice would be no better than a simple QoS priority queue.

We can break down isolation into three key types:

- Type 1: Resource Isolation
- Type 2: Performance Isolation
- Type 3: Security & Data Isolation

Slice Isolation Levels

Not all isolation is equal. We can define a spectrum of isolation strengths.

- **Logical Isolation (Shared Resources)**: Slices share the same physical and virtual network functions. Isolation is provided by logical mechanisms like QoS schedulers, resource quotas, and virtual routing tables. This is the most common and resource-efficient method.
- **Physical Isolation (Dedicated Resources)**: A slice is given its own dedicated physical hardware resources. For example, a dedicated server blade for its Core NFs and a dedicated fiber wavelength in the transport network. This provides the strongest, most secure isolation but is very expensive and inflexible.

The choice of isolation level is a key design parameter driven by the slice's security and performance requirements.

Type 1: Resource Isolation

This ensures that a slice is guaranteed access to its allocated share of physical resources.

- **RAN:** A slice can be allocated a dedicated set of **Radio Resource Blocks (RBs)**, ensuring it always has air interface capacity.
- **Transport:** A slice can be assigned a dedicated path with guaranteed bandwidth through the transport network using technologies like Segment Routing (SR-MPLS).
- **Core/MEC:** A slice's network functions (UPF, SMF) can be given dedicated virtual CPU cores, RAM, and storage on the underlying cloud infrastructure.
- **Mechanism:** Achieved through a combination of radio resource management, traffic shaping, and hypervisor/container resource pinning.

Type 2: Performance Isolation

This is the direct result of resource isolation. It guarantees that the **performance** of one slice is not affected by others.

This solves the classic **"noisy neighbor" problem**.

Scenario: The "Best-Effort Internet" slice is suddenly hit with a massive DDoS attack or a viral video causes a huge traffic surge.

The Slicing Guarantee: Due to resource isolation, this traffic storm is contained within its own slice's resource limits. The guaranteed throughput and latency of the "URLLC Automotive" slice and the "mMTC Smart Meter" slice are **completely unaffected**.

Type 3: Security & Data Isolation

This ensures that traffic from different slices is kept separate and that a security breach in one slice cannot compromise another.

Mechanisms:

- **Traffic Separation:** Using virtualization techniques like VLANs or VXLANs to create separate logical networks for each slice.
- **Function Isolation:** Deploying dedicated network function instances (e.g., a separate UPF) for each slice, rather than sharing them.
- **Data Isolation:** Ensuring that slice-specific data (like subscriber information or monitoring data) is stored in separate, access-controlled databases.
- **Management Isolation:** A customer for Slice A should only be able to see and manage the resources and performance of their own slice

Slice Elasticity

A network slice is not static. A key feature is **elasticity**, the ability to scale resources up or down based on demand, managed by the orchestration system.

Example: An eMBB slice for a football stadium.

- **Baseline:** Consumes minimal resources during the week.
- **Match Day:** The orchestrator automatically scales up the slice's resources (more RAN PRBs, more UPF capacity) just before the match starts.
- **Post-Match:** The resources are automatically scaled back down.

This allows operators to use their infrastructure efficiently, allocating capacity only when and where it is needed.

End-to-End Slice Architecture

We have established the "what" and "why" of network slicing. Now, we will dissect the "how."

A network slice is an **end-to-end** concept. This is not just a marketing term; it has a precise technical meaning.

We will break down how a slice is technically realized across the three major domains of a mobile network:

- **The Radio Access Network (RAN)**
- **The Transport Network (TN)**
- **The Core Network (CN)**



The End-to-End Principle in Network Slicing

A slice's performance guarantee (e.g., an end-to-end latency of 10ms) is like a chain: it is only as strong as its weakest link.

It's useless to build an ultra-low latency Core Network if the Radio Access Network introduces 50ms of jitter, or if the Transport Network in between is congested.

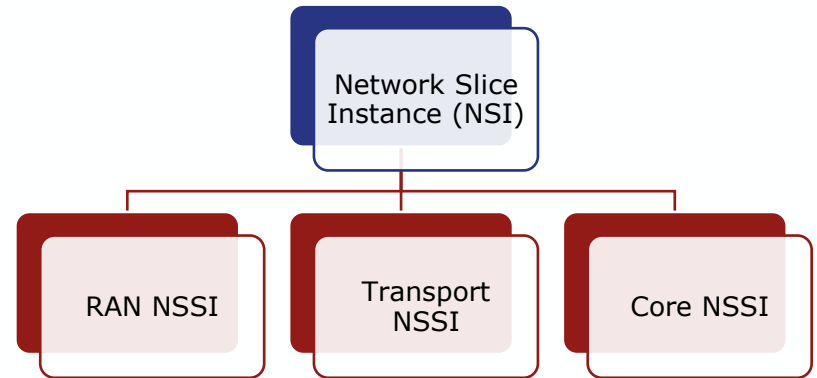
Therefore, a true network slice requires **resource allocation and feature customization across every single domain** that the user's traffic will traverse, from the antenna to the application server.

High-Level Slice Architecture

To manage the complexity of an end-to-end slice, the 3GPP architecture breaks it down into a hierarchical structure.

Network Slice Instance (NSI): This is the complete, end-to-end logical network that is exposed to the customer. It represents the deployed slice.

Network Slice Subnet Instance (NSSI): An NSI is constructed by combining one or more NSSIs. An NSSI represents the portion of the slice's resources and functions within a **specific technical domain**.



The Network Slice Instance (NSI)

The NSI is the realization of a network slice. When an operator "deploys" a slice for a customer, they are creating an NSI.

It is composed of a set of managed network functions and allocated resources (compute, storage, networking).

The NSI is what is monitored against the Service Level Agreement (SLA). For example, the SLA might state that the "URLLC NSI" must provide 99.999% reliability.

An NSI can be shared among multiple tenants or be dedicated to a single one

The Network Slice Subnet Instance (NSSI)

The NSSI represents the slice's footprint within a single administrative or technological domain. The most common domains are:

- **RAN NSSI:** The radio resources (e.g., Radio Bearers, scheduling configuration) and RAN functions (e.g., CU/DU) allocated to the slice.
- **Transport NSSI:** The bandwidth and routing paths allocated to the slice in the transport network that connects the RAN to the Core.
- **Core NSSI:** The set of Core network functions (e.g., AMF, SMF, UPF) and resources that constitute the slice's mobile core.

Orchestrating an end-to-end NSI requires coordinating the creation and configuration of these constituent NSSIs across different management domains.

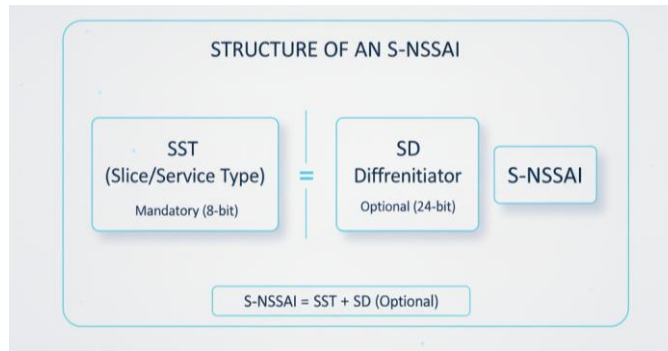
Slice Identification: The S-NSSAI

For the network to manage and route traffic for different slices, it needs a standardized identifier.

The **S-NSSAI (Single Network Slice Selection Assistance Information)** is the unique ID for a network slice.

The S-NSSAI is composed of two parts:

- **SST (Slice/Service Type)**: A mandatory field that identifies the type of service the slice provides
- **SD (Slice Differentiator)**: An optional field that allows an operator to create multiple, distinct slices of the same type



S-NSSAI Values

SST (Slice/Service Type): This is a standardized 8-bit integer. Key standardized values include:

- 1: eMBB (Enhanced Mobile Broadband)
- 2: URLLC (Ultra-Reliable Low-Latency Communications)
- 3: mMTC (Massive IoT)
- 4: V2X (Vehicle-to-Everything)
- 5: HMTC (High-Performance Machine-Type Communications)

SD (Slice Differentiator): This is an optional 24-bit, operator-defined value. It allows an operator to create multiple distinct slices of the same type (e.g., an "eMBB for Enterprise" slice and an "eMBB for Consumers" slice).

Examples:

- An S-NSSAI of {SST=1, SD=100} could be "Premium Consumer Video Streaming".
- An S-NSSAI of {SST=1, SD=200} could be "Enterprise Fixed Wireless Access".
- Both are eMBB slices, but the SD allows the network to differentiate them and apply different policies.

The NSSAI Bundle

A UE can be subscribed to multiple slices simultaneously. The **NSSAI (Network Slice Selection Assistance Information)** is the collection of all S-NSSAIs that a UE is permitted to use.

Key NSSAI Types:

- **Subscribed S-NSSAIs:** The master list of all slices the user is subscribed to. Stored in the UDM (Unified Data Management).
- **Configured NSSAI:** The subset of slices configured on the UE that it can request.
- **Allowed NSSAI:** The subset of slices that the network *actually permits* the UE to use in its current location and registration area. This is determined by the NSSF during the registration procedure.
- **Rejected NSSAI:** Slices the UE requested but was denied access to (e.g., due to local policy or lack of resources).

Radio Access Network Slicing



The RAN Slicing Challenge

The RAN is arguably the most complex and critical domain to slice.

Why? Radio resources are a **scarce, dynamic, and shared medium**. Unlike a fiber optic cable, the wireless channel is subject to interference, fading, and user mobility.

The goal of RAN slicing is to partition these volatile radio resources to provide performance isolation and guaranteed characteristics for different slice types.

RAN Resource Allocation for Slicing

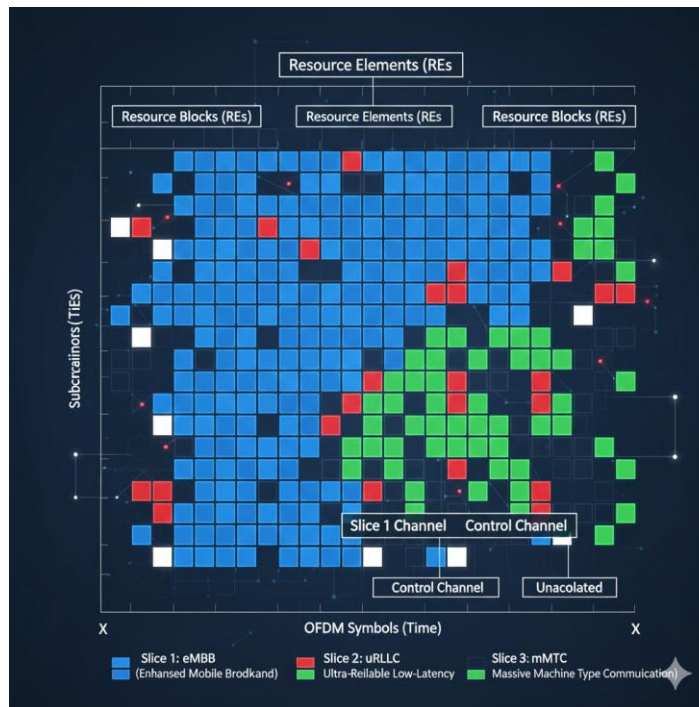
The most direct way to provide isolation in the RAN is to partition the fundamental unit of radio resources: the **Resource Block (RB)**.

Resource Block (RB): The smallest unit of the time-frequency grid in 5G NR that can be allocated to a user.

Strategies for Partitioning:

- **Static Partitioning:** Slice A is permanently allocated 40% of the RBs, Slice B gets 30%, and Slice C gets 30%.
 - **Pros:** Provides very strong, deterministic isolation.
 - **Cons:** Inefficient. If Slice A has no traffic, its resources sit idle and cannot be used by the other slices.
- **Dynamic Partitioning:** Resources are shared, but each slice has a *guaranteed minimum* and a *maximum possible* allocation. Resources are allocated dynamically based on real-time demand.
 - **Pros:** Highly efficient and allows for statistical multiplexing gains.
 - **Cons:** Provides slightly weaker isolation than the static model.

RAN Resource Allocation for Slicing



The RAN Scheduler

The **RAN Scheduler** is a software function in the gNodeB (base station) that makes the microsecond-level decisions about which user gets which Resource Blocks in each transmission time interval (TTI).

In a sliced network, the scheduler must become **slice-aware**.

It must not only consider the channel quality of each user but also the **requirements of the slice** that the user belongs to.

Example:

- A URLLC user's packet arrives. The scheduler must preempt other traffic and grant it resources *immediately* to meet its latency budget.
- An eMBB user's packet arrives. The scheduler might wait for a moment when the user's channel quality is very high to transmit a large burst of data, maximizing throughput.
- An mMTC user's packet arrives. The scheduler might assign it to a very small, robust resource block designed for low-power transmission.



Slice-Aware RAN Scheduling Algorithms

The choice of scheduling algorithm is critical for meeting a slice's SLA.

- **For eMBB Slices:** The scheduler might use a **Proportional Fair (PF)** algorithm. Its goal is to maximize the total cell throughput while ensuring each user gets a fair share of resources over time. It is not latency-sensitive.
- **For URLLC Slices:** The scheduler must use an algorithm like **Earliest Deadline First (EDF)**. Its goal is to service packets with the most urgent deadlines first, even if it means sacrificing overall cell throughput. It is highly latency-sensitive.
- **For mMTC Slices:** The scheduler focuses on efficiently handling a massive number of small, infrequent transmissions and managing device power states.

A slice-aware gNodeB must run multiple scheduling logics in parallel.

Virtualization in the RAN: The CU/DU Split



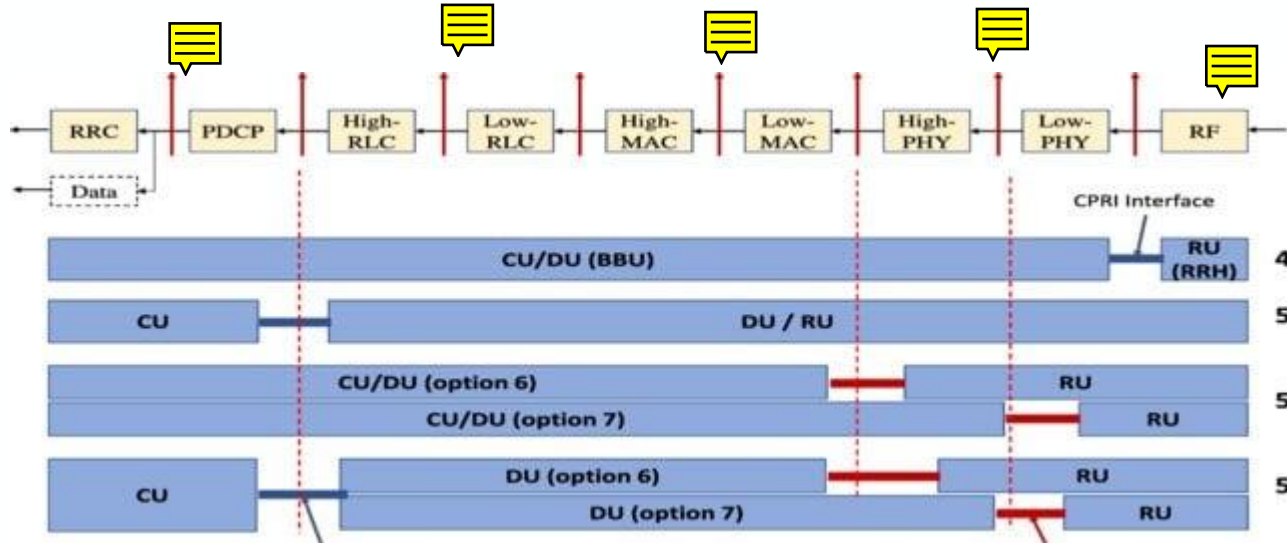
Traditional base stations were monolithic hardware boxes. Modern 5G RANs are disaggregated and virtualized.

The 3GPP architecture splits the gNodeB into three components:

- **Radio Unit (RU):** The physical antenna and radio hardware.
- **Distributed Unit (DU):** Handles the real-time, Layer 1 and Layer 2 radio functions (e.g., modulation, coding). It is latency-sensitive and must be located at the cell site.
- **Centralized Unit (CU):** Handles the less real-time, higher-layer functions (e.g., RRC, PDCP). It is not as latency-sensitive and can be centralized to serve many DUs.

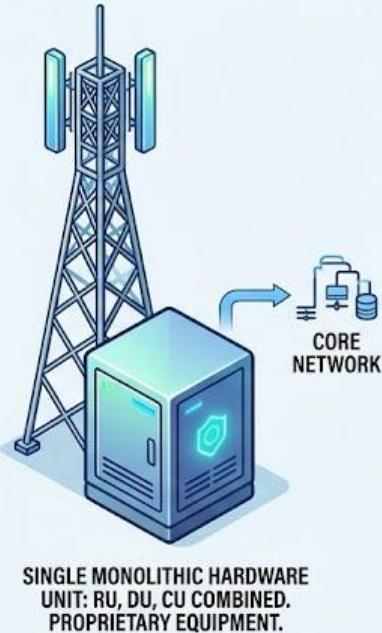
The Slicing Impact: The DU and CU are software functions that can be run as VNFs or CNFs on COTS hardware. This allows for dedicated DU/CU instances to be deployed for different slices, providing strong functional isolation.

CU/DU/RU Split Options



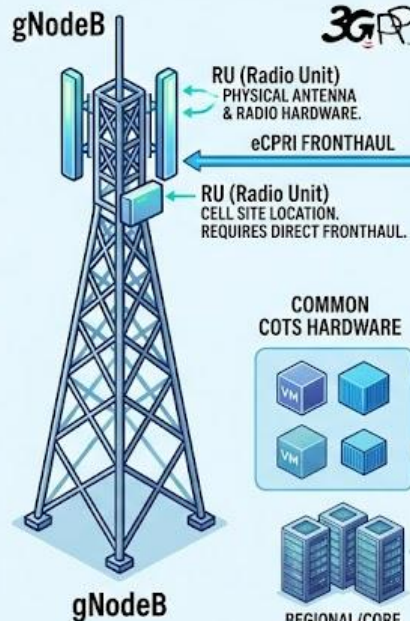
THE EVOLUTION: FROM MONOLITHIC BASE STATION TO DISAGGREGATED, VIRTUALIZED 5G RAN AND SLICING

TRADITIONAL BASE STATION (MONOLITHIC)

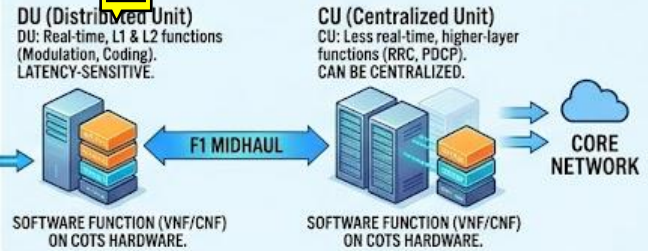


MODERN 5G RAN (DISAGGREGATED & VIRTUALIZED)

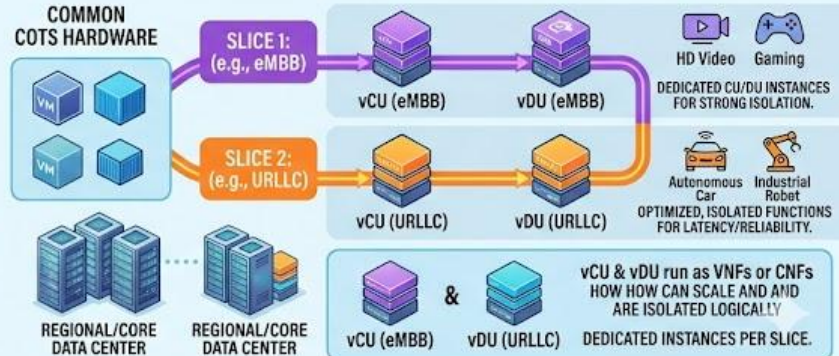
3GPP gNodeB ARCHITECTURE SPLIT



THE SLICING IMPACT



COMMON COTS HARDWARE



RAN Slicing Implementation Models

Model A: Shared RAN, Slice-Aware Scheduler: A single set of CU/DU functions serves all slices. The scheduler is the only component that is slice-aware. This is the simplest but provides the weakest isolation.

Model B: Shared CU, Dedicated DU: The CU is split into a shared Control Plane part (CU-CP) and **dedicated User Plane parts (CU-UP)** for each slice, while the DU is fully dedicated. This provides isolation for user data processing.

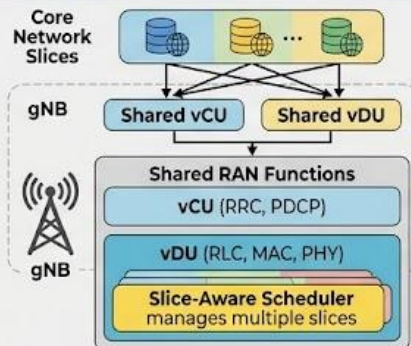
Model C: Fully Dedicated RAN Functions: A fully dedicated vCU and vDU instance is deployed for each slice. This provides the strongest possible isolation but is the least resource-efficient.



COMPARISON OF RAN SLICING MODELS: ISOLATION VS. EFFICIENCY

Illustrating three conceptual models for implementing **Network Slicing** in the Next-Generation RAN (NG-RAN) based on functional gNB split (CU/DU) and **resource allocation**.

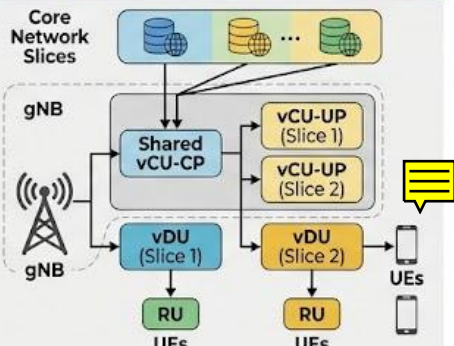
MODEL A: SHARED RAN, SLICE-AWARE SCHEDULER



Weakest Isolation
Highest Resource Efficiency
Scheduler is the only slice-aware component.

Isolation Level	Low	Highest
CU Functions	Shared	Shared
DU Functions	High	Shared
Scheduler	Shared	Slice-Aware
Complexity	Quality	Highs
Efficiency	Highest	Highs

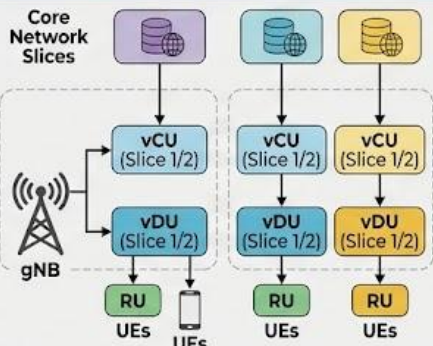
MODEL B: SHARED CU-CP, DEDICATED CU-UP & DU



Isolation for User Data Processing
Medium Resource Efficiency
Shared control, dedicated user plane.

Isolation Level	Low	Medium
CU Functions	Shared	Medium
DU Functions	Shared	Medium
Scheduler	Shared	Shared
Complexity	Quality	Control
Efficiency	High	Highest

MODEL C: FULLY DEDICATED RAN FUNCTIONS



Strongest Possible Isolation
Least Resource Efficient
Fully dedicated RAN instances per slice.

Isolation Level	Low	Highest
CU Functions	Shared	Shared
DU Functions	Shared	Shared
Scheduler	Shared	Shared
Complexity	Quality	Highs
Efficiency	Highest	Least

Transport Network Slicing



Slicing the Transport Network (TN)

The **Transport Network (TN)** is the "middle mile" of the network. It's the IP/MPLS network that provides the connectivity between the hundreds of RAN sites and the centralized Core Network data centers.

The TN is often called the **xHaul** network (Fronthaul, Midhaul, Backhaul).

The Challenge: The TN must be able to provide differentiated paths and guaranteed bandwidth for the traffic from different RAN slices. A best-effort IP network cannot provide these guarantees.

Transport Slicing: Hard vs. Soft Slicing

Hard Slicing: Provides the strongest isolation by dedicating physical or logical resources.



- **Mechanism:** Assigning slice traffic to specific wavelengths in a DWDM system, or to specific time slots in a FlexE (Flexible Ethernet) interface.
- **Pros:** Deterministic performance, completely isolated.
- **Cons:** Inflexible and inefficient.

Soft Slicing: Provides logical separation over a shared packet network. This is the more common and flexible approach.

- **Mechanism:** Using Quality of Service (QoS) and traffic engineering techniques to create virtual tunnels with performance guarantees.
- **Pros:** Efficient, flexible, and scalable.
- **Cons:** Provides statistical guarantees, not deterministic ones.

Soft Slicing Technology: Network Overlays (VPNs)

We can create logically separate virtual networks for each slice by creating **VPNs** across the shared transport infrastructure.

Each slice is mapped to its own VPN, which provides traffic separation.

Technologies:

- **VLANs**: Simple Layer 2 separation. Does not scale well.
- **IPsec/GRE Tunnels**: Layer 3 tunnels. Can be complex to manage at scale.
- **MPLS-based L3VPNs / L2VPNs**: The traditional, highly scalable, and robust way that service providers have offered VPN services for decades. This is a very mature technology.

Multiprotocol Label Switching (MPLS) is a technique allowing to route data from one node to the next **based on labels** rather than network addresses. MPLS labels identify established paths between endpoints. MPLS can encapsulate packets of various network protocols and supports a range of access technologies.

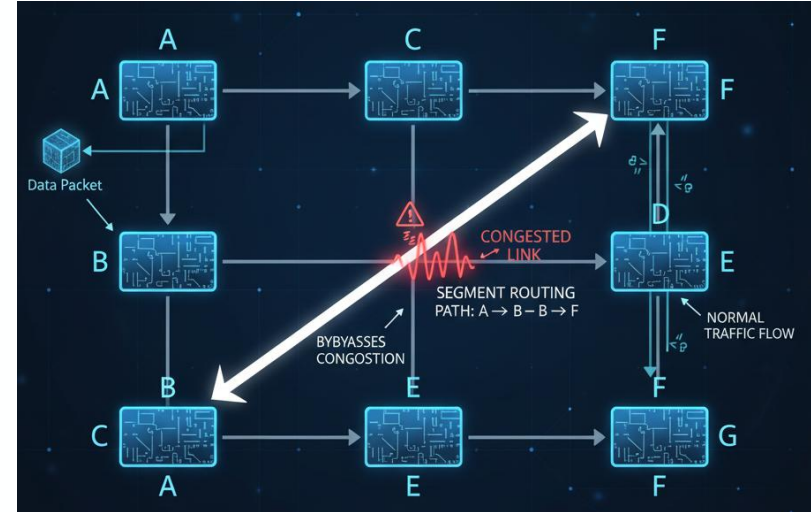
Soft Slicing Technology: Traffic Engineering



Creating tunnels is not enough; we need to steer them along paths that can meet the slice's performance requirements. This is **Traffic Engineering (TE)**.

Technologies:

- **RSVP-TE**: A traditional protocol for signaling TE paths in an MPLS network. It is stateful and complex.
- **Segment Routing (SR)**: The modern approach. It is a more scalable and flexible TE technology. SR allows the ingress router to "steer" a packet along a specific, explicit path by prepending a list of "segments" (instructions) to the packet header.



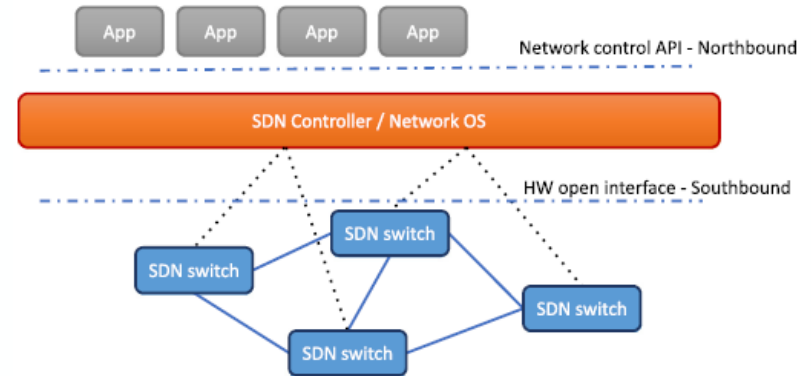
The Role of the Transport **SDN Controller**

The Transport Network is managed by a dedicated **Transport SDN Controller**.

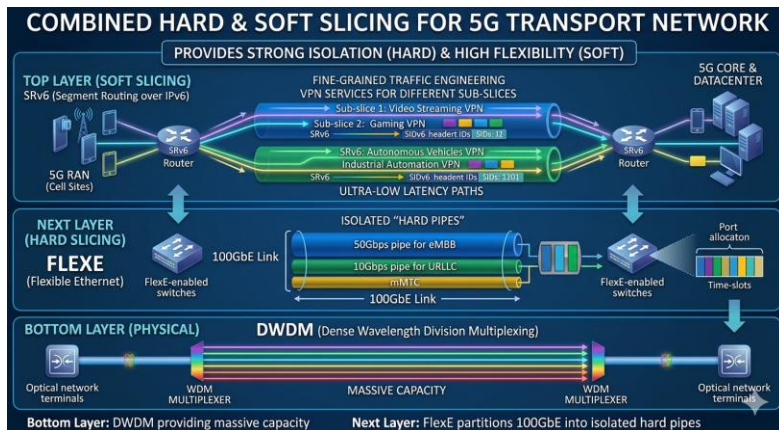
The **Transport NSSMF** communicates with this controller.

Workflow:

1. The Transport NSSMF receives a request: "Create a 1Gbps, low-latency path for the URLLC slice from gNodeB-10 to UPF-5."
2. The NSSMF translates this into an intent for the **SDN Controller**.
3. The SDN Controller calculates the optimal explicit path across the physical network that meets the latency and bandwidth constraints.
4. It then programs the ingress router with the correct SRv6 segment list to enforce this path.



A Modern Transport Slicing Architecture



A state-of-the-art transport network for 5G slicing combines both hard and soft slicing.

A layered architecture:

- **Bottom Layer (Physical):** DWDM providing massive capacity.
- **Next Layer (Hard Slicing): Flexible Ethernet (FlexE)** is used to partition a physical 100GbE link into smaller, isolated "hard pipes" (e.g., one 50Gbps pipe for eMBB, one 10Gbps pipe for URLLC).
- **Top Layer (Soft Slicing): Segment Routing over IPv6 (SRv6)** is used within each FlexE pipe to provide fine-grained traffic engineering and VPN services for different sub-slices.
- This combination provides both strong isolation and high flexibility.

Core Network Slicing



Slicing the Core Network (CN)

We have successfully sliced the Radio and Transport domains. The final domain in our end-to-end journey is the **Core Network**.

The 5G Core is fundamentally different from its predecessors. It was designed from the ground up based on cloud-native principles, making it inherently more flexible and amenable to slicing.

We will explore:

- The key architectural enablers in the 5G Core.
- The mechanisms for selecting and isolating slices.
- The lifecycle and management of core network functions for a slice.

Architectural Enabler: The 5G Service-Based Architecture (SBA)

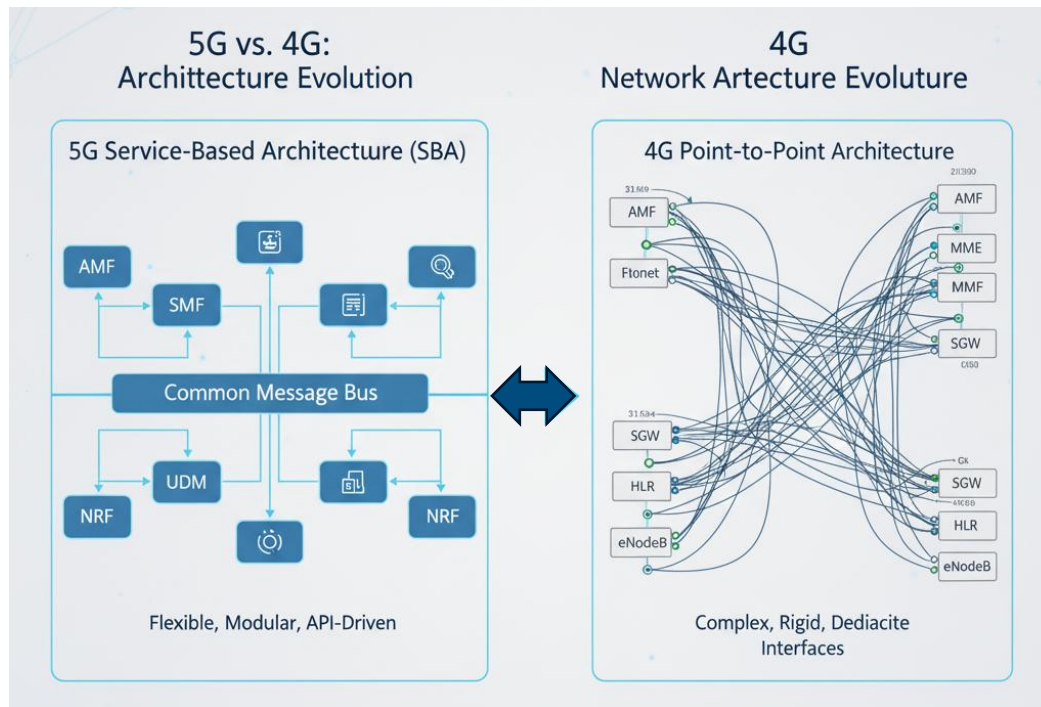
The 4G Core (EPC) was based on a point-to-point architecture with specialized protocols.
The 5G Core is a **Service-Based Architecture (SBA)**.

Key Principles:

- **Network Functions as Microservices:** Each Core Network Function (e.g., AMF, SMF) is implemented as a software microservice, often running in a container.
- **Service-Based Interfaces:** Network Functions communicate with each other using standard, web-scale RESTful APIs (HTTP/2 and JSON) **over a shared message bus**.
- **Stateless Functions:** Functions are designed to be as stateless as possible, storing their state in a centralized database (the UDR - Unified Data Repository).

Impact on Slicing: This cloud-native design makes it incredibly easy to **instantiate, chain, and scale dedicated network functions for a specific slice**. You can deploy a dedicated SMF for the "Automotive" slice just like you would deploy any other microservice.

Architectural Enabler: The 5G Service-Based Architecture (SBA)



Core Network Slicing: Implementation Mechanisms

How do we create an isolated Core Network for a slice? There are two primary approaches, which can be mixed and matched:

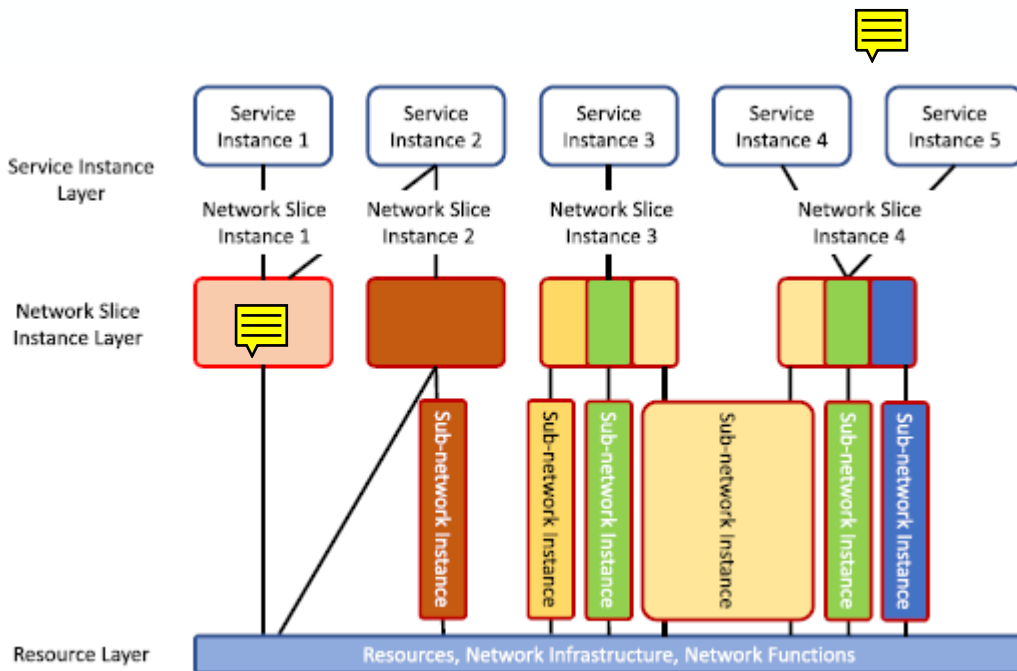
Option 1: A Completely Dedicated Core Network Instance

- A full set of Core Network Functions (AMF, SMF, UPF, etc.) is deployed and dedicated to a single slice.
- **Pros:** Provides the **strongest possible isolation** (security, performance, and functional).
- **Cons:** Highest resource cost. Typically reserved for very high-value, critical slices (e.g., a slice for national emergency services).

Option 2: Sharing of some NFs, Dedication of others

- Some functions that are common across all slices (like the AMF, which handles initial device registration) can be shared.
- Other functions that are slice-specific (like the UPF, which handles user traffic and QoS) can be dedicated.
- This provides a practical balance between isolation and resource efficiency. This is the most common model.

Dedicated vs Shared Slices



Control Plane vs. User Plane Isolation

It is critical to distinguish between isolating the control and user planes.

Control Plane (CP) Isolation:

- **Functions:** AMF, SMF, PCF, etc.
- **Impact of Sharing:** A software bug or a signaling storm (e.g., from millions of IoT devices connecting at once) in one slice could crash a shared AMF, affecting the control plane of all slices using that AMF.
- **Benefit of Dedication:** A dedicated AMF or SMF for a critical slice ensures its control plane is completely insulated from failures or load in other slices.

User Plane (UP) Isolation:

- **Function:** UPF.
- **Impact of Sharing:** A DDoS attack or a massive video download in one slice could consume the forwarding resources of a shared UPF, causing packet loss and increased latency for all other slices using that UPF.
- **Benefit of Dedication:** A dedicated UPF provides guaranteed throughput and latency.



Slice-Aware Functions in the 5G Core



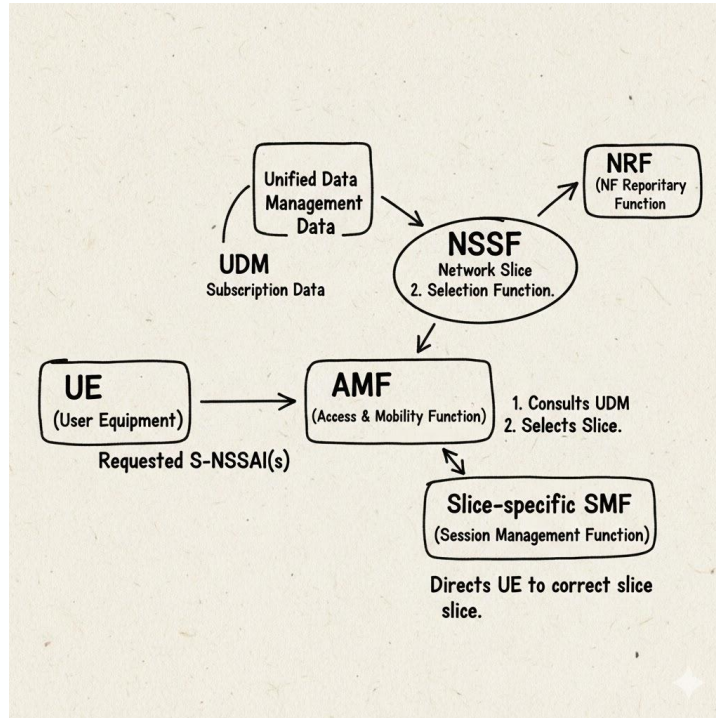
Network slicing in the 5G Core is achieved through the coordinated operation of multiple control and data plane functions, each contributing to slice selection, management, and service delivery.

- **AMF** supports **access and mobility management** and participates in **slice selection** during UE registration.
- **NSSF** determines the **most suitable slice instance** for the UE based on subscription, policies, and requested services.
- **SMF** manages **PDU sessions** and associates them with the selected slice.
- **UPF** provides **user-plane forwarding** and can be instantiated or selected per slice to meet specific latency, throughput, or locality requirements.
- **NRF / UDM / PCF** support slice-aware operation by enabling **NF discovery, subscription handling, and policy control**.

A 5G slice is not usually a completely separate core network; rather, it is an **end-to-end logical network built through the selective sharing or dedication of core functions and resources**.

5GC slicing combines common control functions with slice-specific policies, sessions, and user-plane treatment to deliver differentiated services.

High-Level Overview Slice Selection Procedure



Slice selection is the procedure by which a UE is associated with the correct Network Slice Instance(s) when it registers with the network.

This is a sophisticated, policy-driven process orchestrated by several different 5G Core functions.

A simplified process:.

- A **UE** sends its requested S-NSSAIs.
- The **AMF** receives the request.
- The AMF consults the **NSSF**.
- The NSSF consults the **NRF** and the user's subscription data (from the **UDM**).
- The AMF then directs the UE to the correct slice-specific **SMF**.

Variants of Core-Network Slicing

So far, we have described CN slicing from the reference 5G architecture viewpoint. In practice, however, slices can be deployed in different ways depending on:

- **Ownership model:** single operator, shared infrastructure, or multiple administrative domains
- **Tenancy and coordination:** different actors may share resources while preserving isolation and policy control
- **Function placement:** core functions can be deployed centrally or moved closer to the edge
- **Service requirements:** latency, scalability, locality, and operational constraints influence deployment choices

The logical concept of a network slice remains the same, but its **implementation and operation may vary significantly** depending on business and technical requirements.

These deployment variants directly affect **isolation, performance, orchestration complexity, and support for edge-enabled services.**

Single Owner, Single Controller

Direct Network Slicing Functionalities:

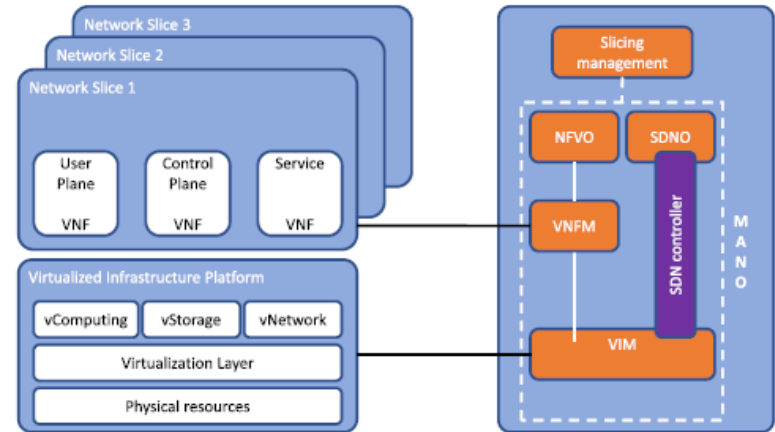
- SDNs enable direct network slicing functionalities.
- The SDN environment provides an abstraction of network resources through the Northbound interface of the SDN controller.

Management and Orchestration:

- Implemented on top of the SDN controller by exploiting the Northbound interface.
- The SDN controller operates as the SDN Orchestrator.

Architecture:

- Depicts the SDN controller functioning as the SDN Orchestrator.



Single Owner, Multiple Tenants – SDN Proxy



Supporting Multiple Virtual Networks:

- Common in data centers and service provider networks.

SDN Proxy:

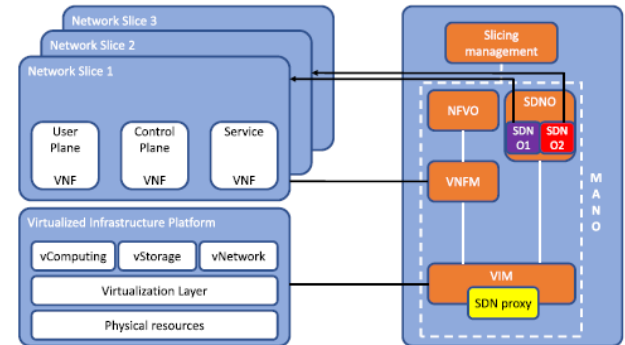
- Controlled by the owner of the physical infrastructure.
- Provides an abstraction of the network forwarding path.
- Allows the SDN proxy to slice the network.

Functionality:

- Uses the SDN protocol to define a hardware abstraction layer.
- Sits between control and forwarding paths on a network device.
- Enforces rules and agreements defining network slices.
- Maintains isolation between slices.

Architecture:

- Depicts the SDN proxy's role in network slicing.
- The SDN proxy provides topology and capacity slicing to the orchestrators of the different slices (SDNO1 and SDNO2).

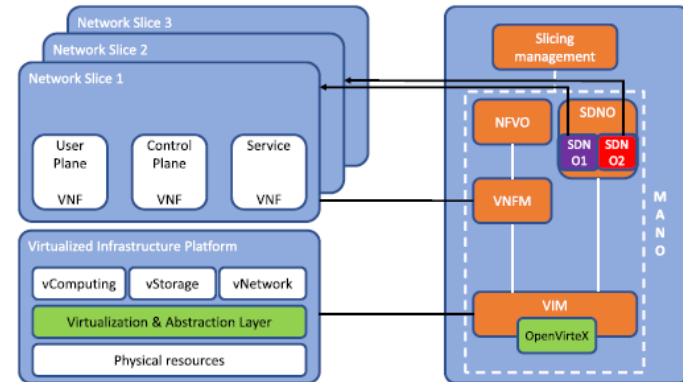


Multiple Owners, Multiple Tenants in Network Slicing

Allows tenants to specify their desired topology and addressing.

Enables infrastructure owners to retain control of their own virtual SDN network.

The SDN orchestrators have more degrees of freedom in defining topology and configuration on their slices.



Edge Computing for Network Slicing

Localized Processing: Edge computing processes data at the edge of the network, reducing the need to send data back to a centralized cloud

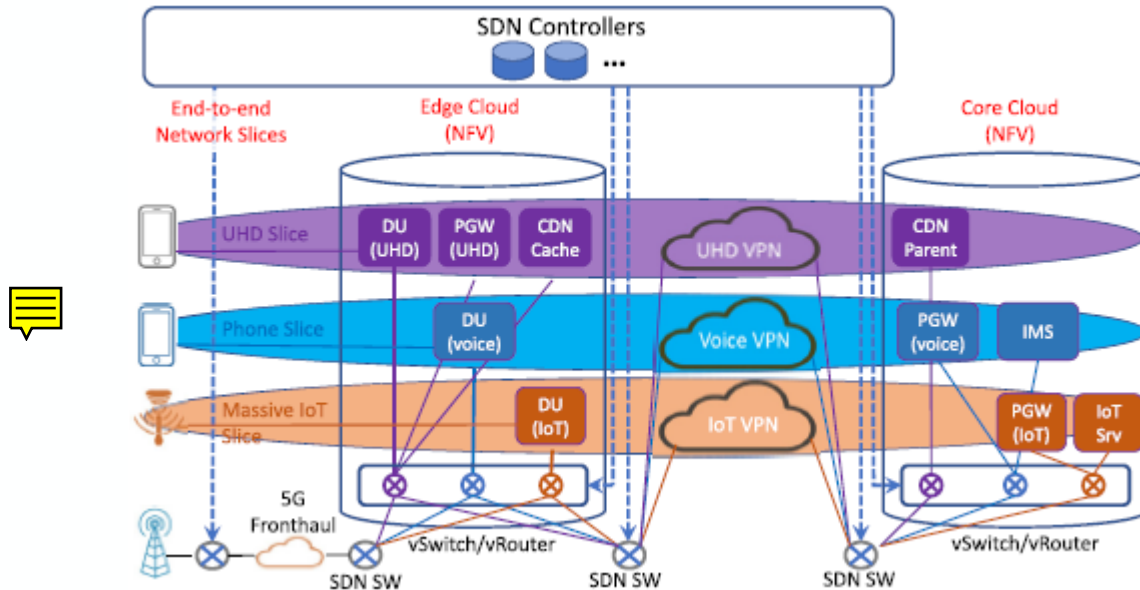
Enhanced Performance: By processing data closer to the end-user, edge computing ensures lower latency and higher reliability, which are critical for network slices with stringent performance requirements

Dynamic Resource Allocation: Edge computing enables dynamic allocation of resources to different network slices based on real-time demand and conditions

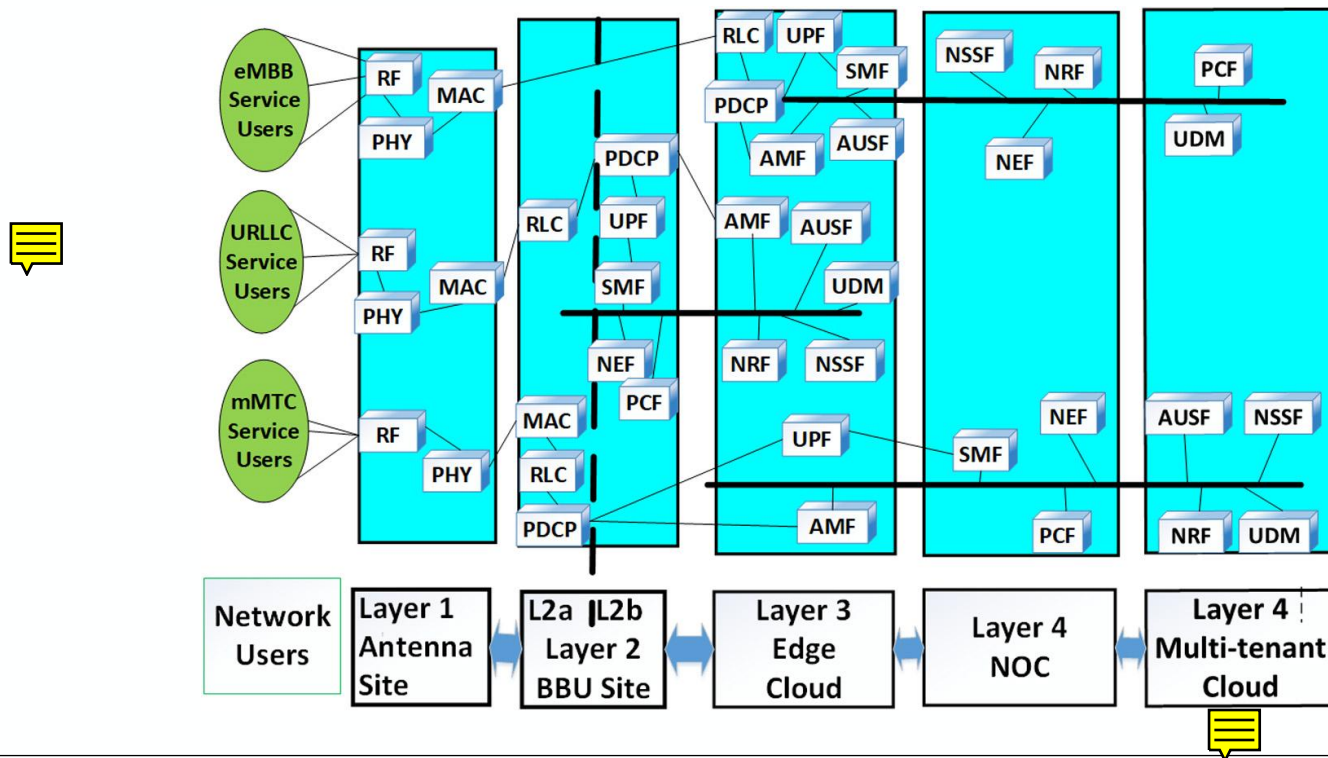
Benefits of Combining Edge Computing with Network Slicing:

- Optimized Resource Utilization: Efficiently utilizes network resources by allocating them to where they are needed most
- Improved User Experience: Provides faster and more reliable services to end-users by minimizing latency and maximizing bandwidth
- Scalability: Supports the scalability of 5G networks by enabling the creation of multiple, isolated network slices for different use cases

Detailed example of the deployment of network slices in 5G



A possible network functions placement



End-to-End Slice Management & Orchestration



End-to-End Slice Management & Orchestration

We have now designed a complete end-to-end network slice across the RAN, Transport, and Core domains.

But how are these complex, multi-domain logical networks actually created, monitored, and decommissioned?

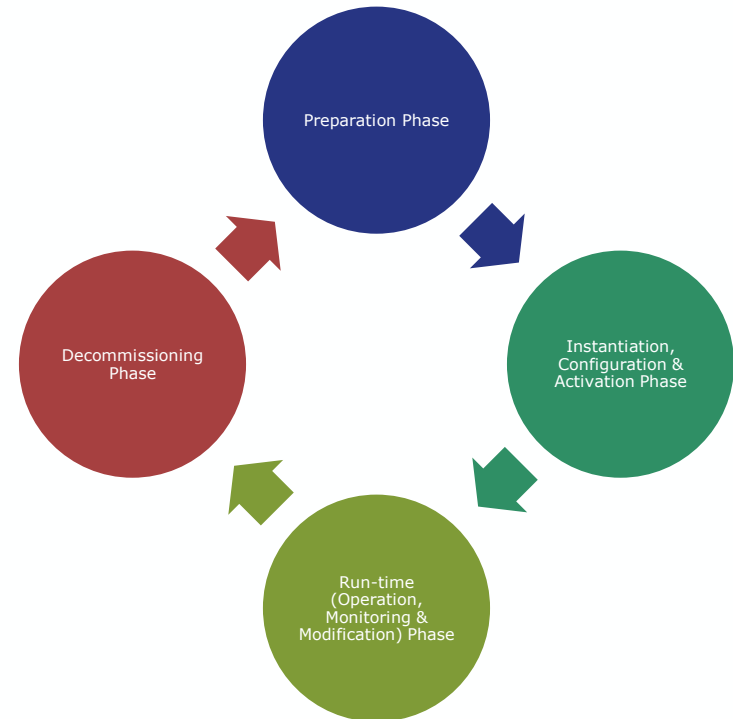
We will dive into the **management and orchestration (MANO)** framework for network slicing, as defined by 3GPP and GSMA.

We will explore the key functional components and the end-to-end workflows that enable **on-demand, automated network slicing**.

The Network Slice Lifecycle

A network slice, like any service, has a well-defined lifecycle. The orchestration framework is responsible for managing the slice through all of these phases.

- **1. Preparation Phase:** The operator designs the slice templates, prepares the network resources, and onboards the required network functions. This is done before any customer orders a slice.
- **2. Instantiation, Configuration & Activation Phase:** A new Network Slice Instance (NSI) is created based on a customer order. Resources are allocated, functions are deployed and configured, and the slice is activated.
- **3. Run-time (Operation, Monitoring & Modification) Phase:** The active slice is now carrying traffic. Its performance is continuously monitored against the SLA. The orchestrator can modify the slice's capacity or topology in response to changing demands (e.g., scaling). This is also known as a "closed-loop" assurance.
- **4. Decommissioning Phase:** When the slice is no longer needed, it is gracefully deactivated, and all of its allocated resources are released back into the shared pool.



The Management Roles & Actors

To understand the management architecture, we first need to define the roles of the different business entities involved.

- **The Customer (or Communication Service Customer)**: The enterprise or vertical industry (e.g., a car manufacturer, a broadcaster) that consumes the network slice. They order the slice and have an SLA defining its performance.
- **The Communication Service Provider (CSP)**: The entity that sells the slice to the customer. They provide the customer-facing portal and are responsible for meeting the SLA. This is often the Mobile Network Operator.
- **The Network Operator (NOP)**: The entity that owns and operates the physical network infrastructure (RAN, Transport, Core). This is almost always the Mobile Network Operator.

The Customer (or
Communication
Service Customer)

The
Communication
Service Provider
(CSP)

The Network
Operator (NOP)

The 3GPP Management Architecture

The 3GPP management framework defines three key logical functions that correspond to the business roles we just discussed.

CSMF (Communication Service Management Function):

- **Role:** The "customer-facing" function.
- **Responsibility:** Translates the customer's business requirements into a network slice service order.

NSMF (Network Slice Management Function):

- **Role:** The "end-to-end slice orchestrator."
- **Responsibility:** Manages the entire lifecycle of a Network Slice Instance (NSI) across all network domains.

NSSMF (Network Slice Subnet Management Function):

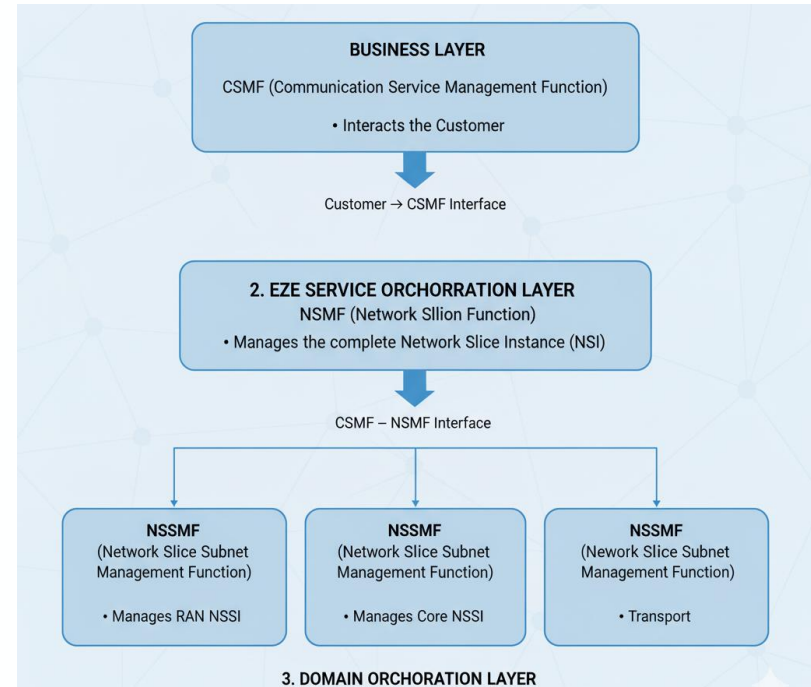
- **Role:** The "domain-specific manager."
- **Responsibility:** Manages the lifecycle of a Network Slice Subnet Instance (NSSI) within a single domain (RAN, Core, or Transport).

A Hierarchical Orchestration Model

These three functions operate in a clear hierarchy, with each layer abstracting the complexity of the layer below it.

- **Top Layer (Business): CSMF** - Interacts with the Customer.
 - *Interface:* Customer -> CSMF
- **Middle Layer (E2E Service Orchestration): NSMF** - Manages the complete NSI.
 - *Interface:* CSMF -> NSMF
- **Bottom Layer (Domain Orchestration): Three NSSMF boxes** (RAN, Core, Transport) - Each manages its own NSSI.
 - *Interface:* NSMF -> NSSMFs

This model allows for a separation of concerns, where the NSMF doesn't need to know the specific details of how radio resources are scheduled, and the RAN NSSMF doesn't need to know about the end-to-end service.



The CSMF: The "Customer Facing Portal"

The CSMF is the bridge between the customer's business intent and the network's technical capabilities.

In practice, this is often implemented as a **web portal** or an **API** for enterprise customers.

Key Functions:

- Exposes a catalog of available slice types (e.g., "Gold URLLC Slice," "Bronze eMBB Slice").
- Receives the customer's order, which includes parameters like:
 - Desired slice type (GST).
 - Geographical area of service.
 - Number of devices to be supported.
 - Specific performance requirements (the SLA).
- Validates that the request is commercially and technically feasible.

The CSMF: Translating Business to Technical

The primary technical role of the CSMF is to translate the customer's high-level request into a formal **network slice service order** that the NSMF can understand.

Example:

- **Customer Input (via Web Form):**
 - Service: "Connected Ambulance Slice"
 - Area: "City of Florence"
 - Performance: "Ultra-Reliable"
 - Devices: "50 Ambulances"
- **CSMF Output (API call to NSMF):**
 - create_slice_request
 - slice_template: URLLC-GST
 - sla_parameters: latency=5ms, reliability=99.999%, area_list=[florence_cell_ids]
 - ue_count: 50

The Service Level Agreement (SLA)

Content:

- The SLA is the formal contract between the customer and the provider that defines the guaranteed performance of the slice.
- It is the single most important document defining the slice's behavior.
- **Key Performance Indicators (KPIs) in a Slice SLA:**
 - **Throughput:** Guaranteed minimum bitrate (uplink and downlink).
 - **Latency:** Maximum end-to-end packet delay.
 - **Jitter:** Maximum packet delay variation.
 - **Reliability:** Packet delivery success rate (e.g., 99.999%).
 - **Availability:** Percentage of time the slice is operational (e.g., 99.99%).
 - **Connection Density:** Maximum number of devices per km².
- The job of the entire orchestration stack is to ensure these SLA KPIs are met.

The NSMF: The "End-to-End Slice Orchestrator"

The NSMF is the **master orchestrator for the Network Slice Instance (NSI)**.

It receives the service order from the CSMF and is responsible for the entire end-to-end lifecycle of that slice.

Key Functions:

- **Decomposition:** Decomposes the end-to-end NSI request into requirements for each domain (RAN, Transport, Core).
- **Coordination:** Interacts with the NSSMFs for each domain to request the allocation of resources and the configuration of functions.
- **End-to-End Monitoring:** Continuously collects KPIs from each domain to monitor the E2E health of the slice against its SLA.
- **Lifecycle Management:** Manages the instantiation, modification, and decommissioning of the NSI.

The Blueprint: The Network Slice Template (NST)

While the CSMF uses a generic template (GST), the NSMF uses a much more detailed, operator-specific **Network Slice Template (NST)**.

The NST is an internal blueprint that contains all the technical information needed to deploy a specific type of slice on the operator's network.

Contents of an NST:

- Which specific Network Functions (or microservices) are needed.
- The VNF/CNF descriptors for those functions.
- The topology of how the functions should be chained together (the VNFFG).
- The specific resource requirements for each domain (e.g., how many RBs in the RAN, which UPF to use in the core).

The NSSMF: The "Domain-Specific Manager"

The NSSMF is the "expert" responsible for managing a slice within a **single technology domain**.

There are separate, specialized NSSMFs for the RAN, Transport, and Core.

Key Functions:

- Receives a request from the NSMF (e.g., "Create a URLLC subnet in the RAN").
- Translates this high-level request into a series of low-level configuration commands for the specific equipment in its domain.
- Manages the lifecycle of the Network Slice Subnet Instance (NSSI).
- Collects domain-specific KPIs and reports them back to the NSMF.

The RAN NSSMF

Responsibilities:

Manages the virtualized RAN functions (vDU/vCU) that are part of the slice subnet.

Configures the RAN scheduler with the appropriate policies for the slice (e.g., a low-latency scheduling policy for a URLLC slice).

Allocates radio resources (e.g., partitions the Resource Blocks) according to the NSSMF's request.

Interacts with the RAN Network Management System (NMS) or the RAN controllers to apply these configurations.

The Core NSSMF

Responsibilities:

Manages the lifecycle of the Core Network Functions (AMF, SMF, UPF, etc.) that are part of the slice subnet. This includes instantiation, scaling, and healing.

Configures the NFs with slice-specific policies.

Manages the selection of which NF instances (e.g., which UPF) will be part of the slice.

Interacts with the NFV Orchestrator (NFVO) and VIM (e.g., OpenStack) to deploy and manage these core functions.

The Transport NSSMF

Responsibilities:

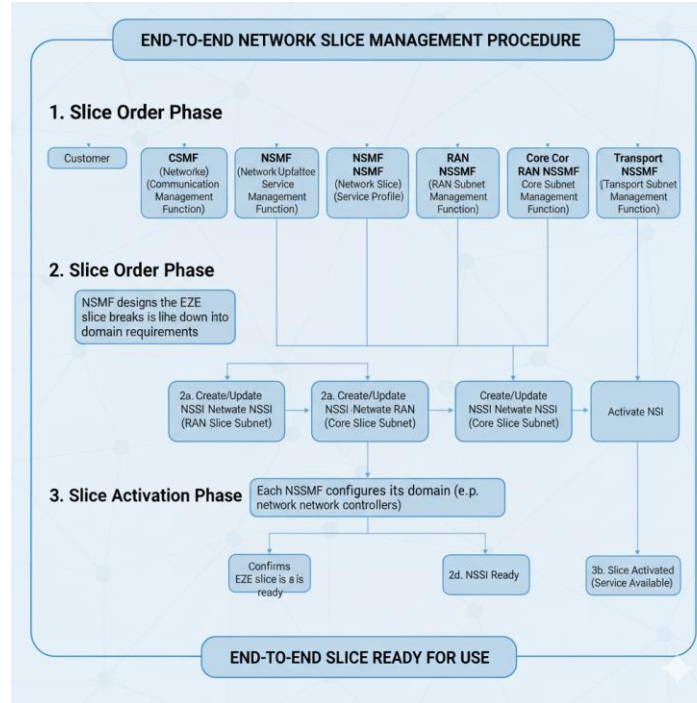
Manages the creation of logical tunnels (e.g., MPLS or Segment Routing paths) across the physical transport network.

Reserves bandwidth for the slice on the transport links.

Configures the routers and switches to enforce QoS policies for the slice's traffic.

Interacts with the Transport Network's SDN Controller or NMS to provision these paths and policies.

The Slice Instantiation Workflow



Step 1: Preparation & Service Order

1. The **Preparation Phase** involves the operator designing and onboarding the NSTs and NSSTs into the orchestrator catalogs.
2. The **Customer** uses a web portal to order a new "URLLC Slice for Factory Robotics." They specify the location, number of robots, and the required SLA (e.g., 5ms latency, 99.999% reliability).
3. The **CSMF** receives this order. It validates the request and translates it into a formal service order containing the technical SLA parameters.
4. The **CSMF** sends a create_NSI_request to the NSMF.

Step 2: E2E Orchestration Begins

1. The **NSMF** receives the request from the CSMF.
2. It selects the appropriate **Network Slice Template (NST)** from its catalog based on the requested GST and SLA.
3. The NSMF now acts as the master orchestrator. It starts the process of building the end-to-end slice by coordinating with the domain managers (the NSSMFs).

Step 3: Core Subnet Creation

1. The **NSMF** sends a create_Core_NSSI_request to the **Core NSSMF**.
2. The **Core NSSMF** interacts with the NFVO/VIM to deploy and configure the necessary Core Network Functions (e.g., a dedicated, edge-deployed UPF and a slice-specific SMF).
3. The Core NFs register themselves with the NRF with the new slice's S-NSSAI.
4. The **Core NSSMF** reports success back to the NSMF.

Step 4: RAN Subnet Creation

1. The **NSMF** sends a create_RAN_NSSI_request to the **RAN NSSMF**.
2. The **RAN NSSMF** interacts with the RAN controller to:
 - Allocate a dedicated set of radio resources.
 - Configure the RAN scheduler with a low-latency profile.
 - Ensure the virtualized RAN functions (vCU/vDU) can support the slice.
3. The **RAN NSSMF** reports success back to the NSMF.

Step 5: Transport Subnet Creation

1. The **NSMF** sends a create_Transport_NSSI_request to the **Transport NSSMF**.
2. The **Transport NSSMF** interacts with the Transport SDN Controller to:
 - Create a dedicated, low-latency Segment Routing path between the specific RAN sites and the specific Core UPF for this slice.
 - Reserve the required bandwidth along this path.
3. The **Transport NSSMF** reports success back to the NSMF.

Step 6: Slice Activation & Monitoring

1. The **NSMF**, having received success confirmations from all three domains, activates the end-to-end **Network Slice Instance (NSI)**.
2. It reports the successful activation back to the **CSMF**, which in turn notifies the customer.
3. The **Closed-Loop Assurance** process begins immediately. The NSMF continuously monitors the E2E KPIs of the slice. If it detects a potential SLA violation (e.g., latency is increasing), it can automatically trigger a modification action (e.g., command the Transport NSSMF to reroute the path, or the Core NSSMF to scale up the UPF).

Putting it all Together: The Core Slice Lifecycle

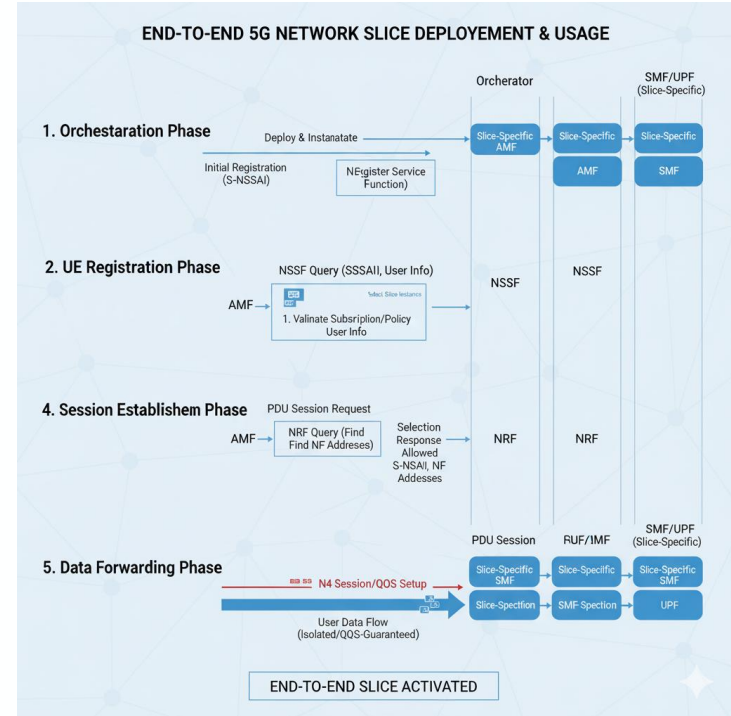
Orchestration Phase: The orchestrator deploys dedicated (or shared) instances of the AMF, SMF, and UPF for a new slice. These new instances register themselves and their supported S-NSSAI with the NRF.

UE Registration Phase: A UE requests the new slice's S-NSSAI.

Slice Selection Phase: The AMF queries the NSSF, which validates the request. The NSSF then queries the NRF to find the correct, slice-specific NF instances.

Session Establishment Phase: The AMF directs the UE's session request to the selected, slice-specific SMF.

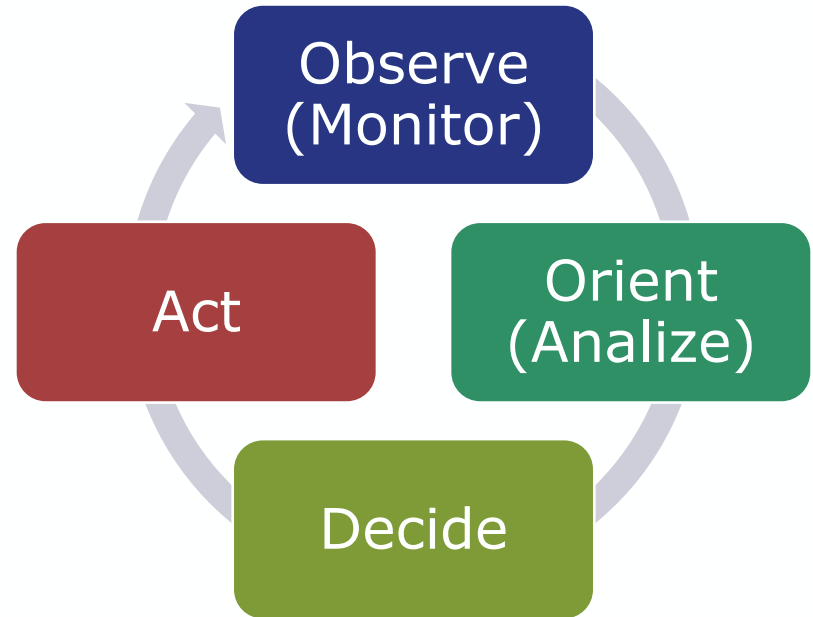
Data Forwarding Phase: The slice-specific SMF programs the slice-specific UPF (via N4) with the correct traffic forwarding and QoS rules for that slice. The user's data now flows through a completely isolated or QoS-guaranteed path in the core.



The Closed-Loop Automation Cycle

This is the "self-driving" aspect of slice management. It follows a continuous loop, often modeled after the military OODA loop:

1. **Observe (Monitor)**: Collect real-time telemetry and KPIs from all NFs and domains in the slice.
2. **Orient (Analyze)**: An analytics or AI engine detects or predicts a potential SLA violation (e.g., "latency is trending upwards on transport link X").
3. **Decide**: The orchestrator's policy engine decides on a remediation action (e.g., "scale out the UPF" or "reroute the transport path away from link X").
4. **Act**: The orchestrator executes the action by sending commands to the relevant NSSMFs/controllers.



Numerical Example & Summary



Slicing in Action & Summary

We have covered the end-to-end architecture and the complex orchestration that underpins network slicing.

To solidify these concepts, we will now walk through a **quantitative, numerical example** of how a physical network's resources are partitioned to support multiple, diverse slices simultaneously.

Finally, we will summarize the key takeaways and future challenges for this transformative technology.

The Scenario: A Shared 5G Infrastructure

The Operator: A mobile network operator owns a single, powerful, virtualized 5G physical infrastructure in a city district.

The Infrastructure Capacity:

- **RAN:** One gNodeB with a total of **100 Physical Resource Blocks (PRBs)** available per scheduling interval.
- **Transport:** A **100 Gbps** physical fiber link connecting the RAN to the Core.
- **Core:** A cloud-native 5G Core. The relevant resources are a **UPF with 200 Gbps** of forwarding capacity and a **compute cluster with 500 vCPUs** available for control plane functions.

The Goal: The operator needs to provision three distinct slices for three different enterprise customers on this shared infrastructure, each with a demanding and unique SLA.

Slice 1: URLLC for a Smart Factory

Customer: An "Industry 4.0" factory that needs to control 500 autonomous robots.

S-NSSAI: SST=2 (URLLC), SD=10

Service Level Agreement (SLA):

- **Latency:** End-to-End latency must be **< 5 milliseconds**.
- **Reliability:** Packet delivery success rate must be **99.999%**.
- **Throughput:** Guaranteed throughput of **1 Gbps** for control traffic.
- **Isolation:** Must have deterministic performance, completely isolated from other network traffic.

Slice 2: eMBB for a Live 4K Sports Broadcast

Customer: A media company broadcasting a live 4K football match from a stadium.

S-NSSAI: SST=1 (eMBB), SD=20

Service Level Agreement (SLA):

- **Throughput:** Guaranteed aggregate downlink throughput of **40 Gbps** for the broadcast video feeds.
- **Reliability:** Packet delivery success rate of **99.99%**.
- **Latency:** End-to-End latency < 50 milliseconds.
- **Isolation:** Must be protected from general public internet congestion in the stadium.

Slice 3: mMTC for a Smart Metering Utility

Customer: A city utility company with 1,000,000 smart meters in the district.

S-NSSAI: SST=3 (mMTC), SD=30

Service Level Agreement (SLA):

- **Connection Density:** Must support **1,000,000 concurrent sessions**.
- **Throughput:** Very low per device (kbps), aggregate < 500 Mbps.
- **Availability:** High availability of the core network functions is critical.
- **Power Consumption:** Must support power-saving features (PSM/eDRX).

Step 1: RAN Resource Partitioning

The **RAN NSSMF** receives the requirements from the NSMF and partitions the 100 available PRBs.

URLLC Slice:

- **Allocation: 10 PRBs Statically Reserved.** To guarantee <5ms latency, this slice gets a dedicated, non-preemptible pool of radio resources. This provides deterministic performance.

eMBB Slice:

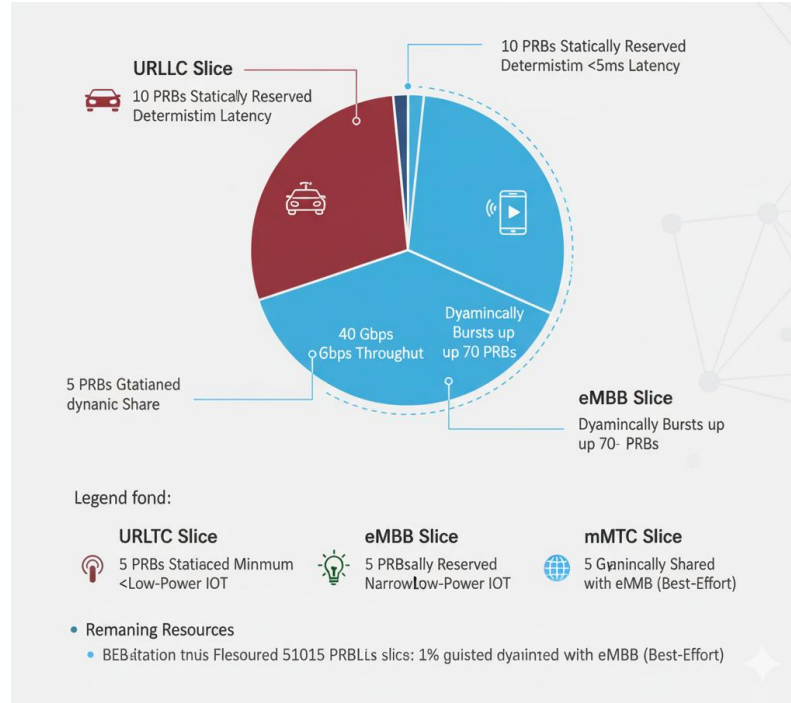
- **Allocation: 10 PRBs Guaranteed Minimum,** with the ability to dynamically burst up to **70 PRBs**. The scheduler will prioritize this slice to meet its 40 Gbps throughput goal, but its resources can be used by others if not needed.

mMTC Slice:

- **Allocation: 5 PRBs Statically Reserved** on a narrow, low-frequency band suitable for wide coverage and low power.

Remaining Resources: 5 PRBs Guaranteed Minimum, dynamically sharing the remaining 70 PRBs with the eMBB slice for best-effort public internet traffic.

Dynamic & Static 5G Resource Allocation



Step 2: Transport Resource Partitioning

The **Transport NSSMF** uses Segment Routing and QoS to create logical pipes on the 100 Gbps link.

URLLC Slice:

- **Allocation:** A **2 Gbps strict-priority queue** on a deterministic, low-latency Segment Routing path. This traffic bypasses all other queues.

eMBB Slice:

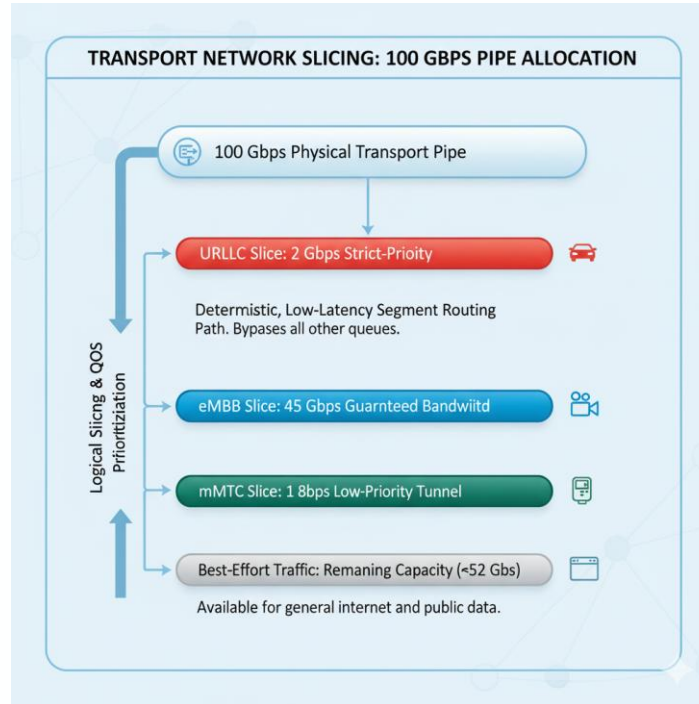
- **Allocation:** A **45 Gbps guaranteed bandwidth tunnel**. This ensures the broadcast feed is never starved for bandwidth.

mMTC Slice:

- **Allocation:** A **1 Gbps low-priority tunnel**.

Best-Effort Traffic: The remaining capacity (~52 Gbps) is available for general internet traffic.

Transport Network Slicing



Step 3: Core Resource Partitioning (User Plane)

The **Core NSSMF** can deploy dedicated or shared UPFs. Let's assume a shared UPF is used, and it enforces slice separation via QoS.

The SMF for each slice sends **QoS Enforcement Rules (QERs)** to the UPF via the N4 interface.

URLLC Slice:

- **QER Config:** Slice-MBR = 1 Gbps, Priority = 1 (Highest).

eMBB Slice:

- **QER Config:** Slice-MBR = 40 Gbps, Priority = 2. (MBR = Maximum Bitrate).

mMTC Slice:

- **QER Config:** Slice-MBR = 500 Mbps, Priority = 5 (Lowest).

This ensures that even on a shared UPF, the forwarding resources are partitioned according to the SLA.

Step 4: Core Resource Partitioning (Control Plane)

The **Core NSSMF** interacts with the VIM to allocate dedicated compute resources for the slice-specific control plane functions.

URLLC Slice:

- **Allocation:** Deploys a dedicated, redundant pair of SMF instances, pinning them to **20 dedicated vCPUs** on hypervisors configured for low-latency NUMA access. This ensures predictable control plane response.

eMBB Slice:

- **Allocation:** Deploys a dedicated, scalable set of SMF instances, allocating **40 vCPUs** in a dynamic pool.

mMTC Slice:

- **Allocation:** Deploys a highly-available cluster of SMFs and other functions optimized for session management, allocating **100 vCPUs** to handle the massive connection state.

The remaining vCPUs are used for shared functions like the AMF, NRF, and best-effort traffic.

Control Plane Load for the mMTC Slice

Let's estimate the control plane needs for the mMTC slice.

Assumptions: 1,000,000 devices. Each device connects once per hour to send a small packet.

Calculation:

- Connections per hour = 1,000,000
- Connections per second = $1,000,000 / 3600 = \sim 278$ connections/sec.

This is the **session establishment rate** that the AMF and SMF for this slice must handle. This is a very different type of load than the high-throughput eMBB slice. The Core NSSMF would use this metric to allocate a specific number of vCPUs to the AMF/SMF instances serving the mMTC slice, optimized for session churn rather than high data throughput.

The Result: End-to-End Resource Partitioning

Slice Type	RAN Allocation (100 PRBs)	Transport Allocation (100 Gbps)	Core UPF (200 Gbps)	Core Control Plane (500 vCPUs)
URLLC	10 PRBs (Static)	2 Gbps (Strict Priority)	1 Gbps (Prio 1)	20 vCPUs (Dedicated)
eMBB	10-70 PRBs (Dynamic)	45 Gbps (Guaranteed)	40 Gbps (Prio 2)	40 vCPUs (Dynamic Pool)
mMTC	5 PRBs (Static NB)	1 Gbps (Low Priority)	0.5 Gbps (Prio 5)	100 vCPUs (Dedicated Cluster)

The Power of Isolation in Action

Scenario: A viral video causes a massive, unexpected traffic surge in the public "Best-Effort Internet" slice, competing for resources with the eMBB slice. The total demand hits 80 Gbps, exceeding the dynamic share of 70 PRBs in the RAN.

The Outcome:

- The eMBB and Best-Effort users experience some congestion.
- The **URLLC slice's performance is completely unaffected**. Its 10 statically reserved PRBs and its strict-priority transport path are untouchable. The factory robots continue to operate with 5ms latency.
- The **mMTC slice is also unaffected**, as it operates on its own dedicated, narrow-band resources.

This is the tangible result and the core value proposition of network slicing.

Network Slicing: Key Takeaways

What it is: A fundamental 5G paradigm for creating multiple, isolated, end-to-end virtual networks on a single physical infrastructure.

Why it matters: It moves operators from selling a single "one-size-fits-all" connectivity product to a flexible **Network-as-a-Service (NaaS)** model for diverse vertical industries.

How it works: By partitioning and dedicating resources across all three network domains—**RAN, Transport, and Core**—using technologies like slice-aware schedulers, Segment Routing, and virtualized core functions.

The Cornerstone: Isolation (resource, performance, security) is the key principle that guarantees an SLA for one slice is not affected by any other slice.

How it's managed: Through a hierarchical, automated orchestration stack (CSMF, NSMF, NSSMF) that manages the entire slice lifecycle.

Final Takeaways

Network slicing is a complex, multi-domain systems engineering challenge. It requires deep expertise across radio, transport, cloud, and automation.

It provides the **necessary network foundation** for many of the advanced applications we have discussed, such as MEC-hosted URLLC services for industry or eMBB services for media.

Slicing allows the network to transform from a rigid, static utility into an agile, dynamic, and programmable platform that can be tailored to the precise needs of any application.

Open Challenges in Network Slicing

End-to-End Monitoring & Assurance: Correlating performance data from three different domains in real-time to monitor an E2E SLA is extremely difficult.

Security: How do we secure the management interfaces and guarantee that a breach in a less secure slice (e.g., best-effort internet) cannot be used to attack the orchestration system or a more critical slice?

Inter-Operator Slicing: How can an operator provide a single, seamless slice for a customer (like a logistics company) whose devices roam across multiple different operator networks globally?

Business Model Adoption: The commercial and operational models for selling and managing slices are still maturing.

Challenge: End-to-End Slice Testing & Validation

Before a slice is sold to a customer, the operator must **certify** that it can meet its SLA. This is a huge challenge.

- **How do you test a URLLC slice?** You need specialized test agents that can generate traffic and measure latency and reliability with microsecond precision across all three network domains simultaneously.
- **How do you test an mMTC slice?** You need to simulate the signaling load of a million devices connecting at once.

Developing these automated, end-to-end testing and validation platforms is a major area of work for network operators.

The Future: AI-Driven & Intent-Based Slicing

The next evolution of slicing is to move towards a zero-touch, AI-driven model.

Intent-Based Networking: The customer will no longer order a slice with technical parameters. They will simply state their business **intent**.

- **Example Intent:** "I am a broadcaster covering the Florence Marathon this Sunday. Ensure all my 50 4K cameras have a flawless connection from these GPS coordinates between 8 AM and 2 PM."

AI-driven Orchestration: An AI engine will take this high-level intent and automatically perform all the steps we discussed:

- Design the optimal slice template (NST).
- Predict the resource demand.
- Trigger the instantiation and configuration of the slice.
- Monitor and autonomously heal the slice in real-time to guarantee the intent is met.



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.2: Network Slicing

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

daniele.tarchi@unifi.it